



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

TESI DI LAUREA MAGISTRALE IN
COMPUTER SCIENCE AND ENGINEERING
INGEGNERIA INFORMATICA

Distributed Streaming Ensemble Methods in Renoir

Authors:

Antonio Sulfaro (259954)
Zhi Qiang Shi (252205)

Advisor:

Prof. Alessandro Margara

Co-advisors:

Prof. Gianpaolo Saverio Cugola
Luca De Martini
Dario d'Abate
Federico Giannini

Academic Year:

2025-2026

Dedicated to my family.

Abstract

The rise of high-velocity data streams in domains such as network monitoring, fraud detection, and the Internet of Things has exposed the fundamental limitations of centralized streaming machine learning frameworks. Existing **Streaming Machine Learning** tools, most notably MOA and River, are constrained by single-threaded execution or JVM-based overhead, making them inadequate for the computational demands of large ensemble methods at scale. This thesis presents a distributed streaming ensemble learning system built on **Renoir**, a high-performance dataflow runtime written in Rust. We design and implement three state-of-the-art ensemble algorithms — **Adaptive Random Forest (ARF)**, **Streaming Random Patches (SRP)**, and **Aggregated Mondrian Forest (AMF)** — within a unified, extensible architecture that decouples the streaming pipeline from the underlying learning algorithms. The system exploits the embarrassingly parallel nature of ensemble methods by distributing individual base learners across worker nodes, using key-based stream routing to coordinate training and prediction aggregation without a centralized bottleneck.

Experimental evaluation across thirteen real-world and synthetic datasets demonstrates that the proposed implementation preserves the predictive accuracy of the original algorithms while delivering decisive throughput advantages at every scale. On a single core, Renoir already outperforms MOA by up to $35.73\times$ and Onelearn by up to $18.24\times$, driven by Rust’s zero-cost abstractions and cache-efficient memory layout. Scaling to 32 cores further widens this gap: our distributed ensembles sustain throughputs that remain an order of magnitude above the centralized competition, confirming that Renoir’s dataflow model translates additional hardware directly into processing speed. This behavior is validated across varying hyperparameter configurations, drift conditions, and a real networked deployment, addressing the full range of practical streaming scenarios.

Keywords: Streaming Machine Learning (SML), ensemble methods, distributed dataflow, distributed systems, ARF, random forest, SRP, AMF, Renoir, Rust

Abstract in lingua italiana

L'ascesa dei flussi di dati ad alta velocità nei domini quali il monitoraggio di rete, il rilevamento delle frodi e l'Internet of Things ha messo in luce i limiti fondamentali dei framework centralizzati di streaming machine learning. Gli strumenti esistenti di **Streaming Machine Learning**, in particolare MOA e River, sono limitati dall'esecuzione a thread singolo o dall'overhead basato su JVM, risultando inadeguati per le richieste computazionali dei metodi ensemble su larga scala.

Questa tesi presenta un sistema distribuito di streaming ensemble learning basato su **Renoir**, un runtime di dataflow ad alte prestazioni scritto in Rust. Progettiamo e implementiamo tre algoritmi di ensemble allo stato dell'arte — **Adaptive Random Forest (ARF)**, **Streaming Random Patches (SRP)** e **Aggregated Mondrian Forest (AMF)** — all'interno di un'architettura unificata ed estensibile che disaccoppia la pipeline di streaming dagli algoritmi di apprendimento sottostanti. Il sistema sfrutta la natura intrinsecamente parallela dei metodi ensemble distribuendo i singoli base learner tra i nodi worker, utilizzando un routing del flusso basato su chiavi per coordinare l'addestramento e l'aggregazione delle predizioni senza un collo di bottiglia centralizzato.

La valutazione sperimentale su tredici dataset reali e sintetici dimostra che la soluzione proposta preserva l'accuratezza predittiva degli algoritmi originali, offrendo al contempo decisivi vantaggi in termini di throughput a ogni scala. Su un singolo core, Renoir supera già MOA fino a $35,73\times$ e Onelearn fino a $18,24\times$, grazie alle astrazioni a costo zero di Rust e a una gestione di memoria efficiente. La scalabilità a 32 core amplia ulteriormente questo divario: i nostri ensemble distribuiti sostengono throughput che rimangono di un ordine di grandezza superiori rispetto alla concorrenza centralizzata, confermando che il modello dataflow di Renoir traduce l'hardware aggiuntivo direttamente in velocità di elaborazione. Questo comportamento è validato attraverso diverse configurazioni di iperparametri, condizioni di drift e una reale implementazione in rete, coprendo l'intera gamma dei flussi di scenari pratici.

Parole chiave: Streaming Machine Learning (SML), ensemble methods, dataflow distribuito, sistemi distribuiti, ARF, foresta casuale, SRP, AMF, Renoir, Rust

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
1 Introduction	1
2 Background and Motivations	3
2.1 Streaming Machine Learning background	3
2.1.1 Streaming Machine Learning	3
2.1.2 Hoeffding Tree	7
2.1.3 ADaptive WINdowing (ADWIN)	10
2.1.4 Adaptive Random Forest	11
2.1.5 Streaming Random Patches	13
2.1.6 Mondrian Tree	17
2.1.7 Aggregated Mondrian Forest	20
2.2 The Renoir Dataflow Runtime	22
2.2.1 Overview and Goals	22
2.2.2 Programming Model	23
2.2.3 Operators	24
2.2.4 Parallel Execution Model	25
2.2.5 Sinks	25
2.3 Motivation	26
3 Design and Implementation	29
3.1 Software Architecture	29
3.1.1 Core Abstraction: Online Learner	29
3.1.2 Configuration and Parameterization	31

3.1.3	Implementation Details and State Management	32
3.2	Distributed Dataflow Pipeline	35
3.2.1	Stage 1: Input and Initialization	36
3.2.2	Stage 2: Per-Tree Processing	37
3.2.3	Stage 3: Per-Instance Aggregation	37
3.2.4	Stage 4: Output and Data Sink	38
3.2.5	Evaluation Test Harness and Post-Processing	39
3.3	Summary	39
4	Experiments and Results	41
4.1	Experimental Setup	42
4.1.1	Hardware Setup	42
4.1.2	Software Setup	42
4.1.3	Datasets Analysis	42
4.1.4	Accuracy Parity Analysis and Validation	45
4.1.5	Reference Frameworks	47
4.2	Experimental Evaluation	48
4.2.1	Comparison with MOA	48
4.2.2	Comparison with River/Onelearn	54
4.2.3	Remote execution	58
4.2.4	Scalability Analysis	61
4.2.5	Comparison with Distributed VHT	67
4.2.6	Hyperparameter Influence	71
4.3	Discussion	77
5	Related work	83
6	Conclusions	87
6.1	Future Works	88
	Bibliography	91
	List of Figures	95
	List of Tables	97
	Acknowledgments	99

1 | Introduction

Modern data-intensive applications do not produce data in batches — they produce it continuously. Network traffic analysis, financial fraud detection, IoT sensor telemetry, and cybersecurity intrusion systems all share a common requirement: decisions must be made in real time, on data that arrives faster than it can be stored and re-examined. Traditional machine learning, which assumes a fixed dataset that can be revisited arbitrarily many times, is structurally incompatible with this reality. The field of **Streaming Machine Learning (SML)** addresses this by enforcing a stricter contract: models must process each observation exactly once, in bounded memory, and must be capable of simultaneously performing continuous online training and instantaneous label prediction at any given moment.

Ensemble methods — and Random Forests in particular — are among the most accurate tools available for streaming classification. They achieve this accuracy by aggregating the votes of many independently trained decision trees, which naturally corrects the bias and variance of any single learner. This independence, however, comes at a cost: maintaining and updating hundreds of trees sequentially on a high-velocity stream creates a severe computational bottleneck. While sequential frameworks such as MOA, SAMOA and River serve as the standard baselines for SML, their execution models remain fundamentally single-threaded or constrained by JVM overhead. Consequently, their maximum processing throughput is easily overwhelmed by the arrival velocity of high-throughput real-world data streams.

The natural solution is horizontal parallelism. Because each tree in an ensemble trains independently, the workload can in principle be distributed across many cores or machines without altering the mathematical behavior of the model. Yet no existing production-grade system exposes this parallelism for ensemble SML in a general, framework-level way. This gap is the central problem this thesis addresses. We present a distributed streaming ensemble learning system built on Renoir, a high-performance dataflow framework written in Rust. We implement three state-of-the-art ensemble algorithms — Adaptive Random Forest (ARF), Streaming Random Patches (SRP), and Aggregated Mondrian

Forest (AMF) — within a unified architecture that exploits the embarrassingly parallel structure of ensemble methods to distribute computation across worker nodes. The goal is twofold: to demonstrate that distributed execution preserves the predictive accuracy of the original algorithms, and to quantify how significantly it exceeds the throughput of centralized alternatives — both on a single machine and in a networked deployment. The contributions of this thesis are the following:

- A general, extensible Renoir-based pipeline for distributed streaming ensemble classification, supporting both Hoeffding-tree and Mondrian-tree families.
- An empirical evaluation showing single-core throughput improvements with accuracy parity across thirteen datasets.
- A scalability analysis demonstrating that distributing across 32 cores further widens the throughput gap over centralized baselines by an order of magnitude, with efficiency consistent with the embarrassingly parallel nature of the problem.
- Evidence that Renoir’s dataflow architecture is particularly well suited to ensemble workloads, amplifying the framework-level performance gain far beyond what is achievable for non-ensemble streaming algorithms.

The remainder of this thesis is organized as follows. Chapter 2 introduces the background concepts — Hoeffding Trees, Mondrian Trees, Ensemble Methods, and the Renoir framework. Chapter 3 describes the system design and implementation. Chapter 4 presents the experimental evaluation and discusses results. Chapter 5 surveys related work. Chapter 6 concludes and outlines directions for future work.

2 | Background and Motivations

The research presented in this thesis is positioned at the intersection of two complementary domains: Streaming Machine Learning (SML) and Distributed Systems. Although our system is an independent implementation built on the Renoir data flow runtime, its architectural core is rooted in the principles of decision trees and ensemble methods such as Streaming Random Patches (SRP), Adaptive Random Forest (ARF), and Aggregated Mondrian Forest (AMF).

This chapter establishes the theoretical and technical basics for the distributed streaming learners detailed in later sections. Section 2.1 outlines the core requirements of SML and the statistical mechanics of Hoeffding Trees and Mondrian Tree, before exploring the ensemble methods central to this work. Section 2.2 summarizes the Renoir programming model, which serves as the engine for the implementation of our distributed forests. Finally, Section 2.3 synthesizes these domains, highlighting our core questions, motivations, and goals driving this research.

2.1. Streaming Machine Learning background

This section provides the necessary background to understand the core algorithms implemented in our system: SRP, ARF, and AMF.

We begin with an overview of SML, defining the environment in which these models operate. This includes a comparison between data stream learning and traditional batch learning, highlighting the divergent underlying assumptions and system requirements. Subsequently, we detail the algorithms themselves, tracing their development from their theoretical foundations to their specific operational mechanics.

2.1.1. Streaming Machine Learning

Streaming Machine Learning (SML) [7, 21] studies how to learn predictive models from data that arrives sequentially over time, potentially without bound. A data stream can be modeled as a sequence of instances (x_t, y_t) , where $x_t \in \mathbb{R}^d$ is a feature vector and y_t

is its corresponding label. Typically, only x_t is immediately available, while y_t may be delayed.

Unlike traditional batch learning, where a fixed dataset is available and can be processed multiple times, SML operates under strict computational constraints. In streaming settings, data is abundant and continuously generated, while memory and processing time are limited. As a result, it is generally infeasible to store all past data or retrain models from scratch. Instead, learning must be performed incrementally, processing each instance efficiently as it arrives.

A streaming learning algorithm is typically required to satisfy the following properties [6]:

- **Single-pass processing:** each instance is processed at most once.
- **Bounded memory:** memory usage must remain limited regardless of stream length.
- **Bounded processing time:** each update must be computationally efficient.
- **Anytime prediction:** the model should be able to produce predictions at any time.

Evaluation in Data Streams. Traditional evaluation methods such as holdout or cross-validation are not suitable in streaming contexts, due to the continuous and potentially infinite nature of the data. Instead, we adopt *prequential evaluation* (Interleaved Test-Then-Train) [18]. The standard processing cycle is therefore: (i) predict the label of the incoming instance, (ii) update the model using the same instance once the true label is available, and (iii) discard the instance. This protocol ensures efficient use of data and provides a continuous estimate of performance over time.

To support this workflow, models rely on compact summaries of the data, known as *sufficient statistics*, instead of storing raw observations [16].

In this work, model quality is assessed in terms of predictive accuracy and computational efficiency (throughput), reflecting the dual objective of maintaining high predictive performance while processing data at high speed.

Concept Drift. A key challenge in streaming environments is *concept drift* [17, 36], where the underlying data distribution changes over time, potentially degrading model performance. In contrast to approaches that assume a stationary distribution, this work

considers methods capable of adapting to such changes through mechanisms such as drift detection and model updates, which are particularly important in ensemble settings.

Data preprocessing. The transition from raw data streams to actionable computational inputs necessitates a robust preprocessing phase to resolve structural inconsistencies. In practical applications, raw data is frequently characterized by missing values or non-numerical attributes that are incompatible with most machine learning estimators. Data imputation serves as the primary corrective measure for missingness, typically involving the substitution of null values with a static numerical default, such as zero.

To handle qualitative attributes, categorical encoding is employed to map non-numerical variables into a format suitable for algebraic manipulation. One-hot encoding is the standard approach for nominal data, transforming categories into sparse binary vectors to avoid the introduction of artificial ordinality. Conversely, label encoding may be utilized to map distinct categories to a sequential integer series. Beyond structural adjustments, feature scaling is required to harmonize variables that span disparate numerical ranges. Techniques such as min-max normalization standardize features within a fixed interval, ensuring that the objective function is not disproportionately biased by variables with larger magnitudes.

Decision Trees in Data Streams

Decision trees are a widely used class of models for classification [8, 33], valued for their interpretability and strong empirical performance. A decision tree consists of internal nodes representing feature-based tests and leaf nodes containing class predictions. Predictions are obtained by routing an instance from the root to a leaf according to the outcomes of these tests.

In streaming scenarios, decision trees are built incrementally: instead of storing instances, the model maintains sufficient statistics at the leaves (e.g., class counts). Each incoming instance is routed to a leaf, and the corresponding statistics are updated.

A central challenge is deciding when to split a node. This is typically addressed using statistical tests, such as the Hoeffding Bound, which determine when sufficient evidence has been accumulated to make a reliable splitting decision. This allows the tree to grow incrementally while approximating the behavior of a batch learner. In contrast, geometric approaches like the restricted Mondrian Process determine splits by computing the absolute distance between a sample and the node’s boundaries.

In this thesis, we extend these principles beyond single-tree models by considering en-

semble methods of them (usually called *forests*), where multiple trees are combined to improve predictive performance and robustness. Additionally, mechanisms for handling concept drift are integrated, enabling the model to adapt to changes in the data distribution over time.

Ensemble Learning Methods

Ensemble methods [14] leverage the predictive power of multiple base learners to produce a single, robust output that typically outperforms any individual constituent model. By aggregating diverse hypotheses, these methods minimize the generalization error and mitigate the risk of overfitting. A quintessential implementation of this paradigm is the **Random Forest** algorithm, which constructs an ensemble of de-correlated decision trees.

The efficacy of an ensemble is fundamentally predicated on **model diversity**. If the individual learners are perfectly correlated, the aggregation process yields no statistical advantage. To induce this diversity, stochasticity is introduced during the training phase through techniques such as:

- **Bootstrap Aggregation (Bagging):** Training models on different random subsets of the training data.
- **Feature Subsampling:** Restricting each split in a tree to a random subset of the total feature space.
- **Mondrian Process:** Making a random cut along a feature chosen with a probability proportional to its spatial distance.

The synthesis of individual predictions into a final result is achieved through various aggregation mechanisms. This work focuses on the following strategies:

- **Majority Voting (Hard Voting):** Each base learner $p_i(x)$ casts a single vote for a class label. The final prediction $P(x)$ is determined by the mode of the distribution:

$$P(x) = \arg \max_j \sum_{i=1}^n \mathbb{I}(p_i(x) = j)$$

where $\mathbb{I}$ is the indicator function.

- **Hard Weighted Voting:** An extension of majority voting where each model's contribution is scaled by a weight w_i , typically derived from its historical performance or validation accuracy.
- **Soft Weighted Voting:** This method aggregates the predicted class probabilities

rather than discrete labels. By calculating the weighted average of the posterior probabilities across all models, this approach accounts for the confidence levels of individual learners, often resulting in higher predictive refinement.

2.1.2. Hoeffding Tree

Hoeffding Trees, also known as Very Fast Decision Tree (VFDT), are a fundamental algorithm for learning decision trees from data streams [15]. They provide a concrete realization of incremental tree induction by combining sufficient statistics with a statistically grounded splitting criterion.

The Hoeffding Bound

The algorithm's ability to learn from streams relies on the **Hoeffding Bound**. It provides a statistical guarantee that a split decision made with a finite sample n is the same as the decision made with infinite data, within a confidence $1 - \delta$, being δ the probability of a large deviation. For a real-valued random variable r with range R , the bound states that after n independent observations, the true mean μ is at least $\bar{r} - \epsilon$ with probability $1 - \delta$, where:

$$\epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}} \quad (2.1)$$

Incremental Split Selection. In a Hoeffding Tree, we use a heuristic $G(X_i)$ (such as Information Gain or Gini Index) to evaluate the merit of a split on attribute X_i .

- Let X_a be the attribute with the highest observed G , and X_b be the second best.
- Let $\Delta G = G(X_a) - G(X_b)$ be the difference between their merits.

If $\Delta G > \epsilon$, the Hoeffding bound guarantees that X_a is the superior split choice with probability $1 - \delta$. If the difference is very small (less than a tie-breaking threshold τ), the split is performed anyway to avoid bottlenecking the tree growth.

To optimize performance, the algorithm employs a **Grace Period** (n_{min}): it only calculates the Hoeffding bound after a minimum number of new instances have reached a leaf, significantly reducing the computational cost of frequent split testing.

Evaluation Metrics. The choice of $G(X)$ depends on the impurity function used:

- **Gini Impurity:** $I_{Gini} = 1 - \sum p_k^2$. Preferred for high-throughput systems due to lower computational overhead.

- **Entropy:** $I_{Ent} = -\sum p_k \log_2(p_k)$. Used to calculate *Information Gain*; offers a more rigorous information-theoretic approach.

Numerical Attribute Handling and Sufficient Statistics

Unlike nominal attributes, where class frequencies per category can be tracked via simple contingency tables, continuous (numerical) attributes present an infinite search space for potential split points. To compute impurity metrics incrementally without storing historical raw instances in memory, leaves must maintain compact structures known as *sufficient statistics*. The two predominant methodologies for managing continuous attributes are Gaussian approximations and dynamic histograms.

Gaussian Approximation. The Gaussian approach assumes that the values for a given attribute X_i , conditioned on a target class y_c , follow a normal distribution $\mathcal{N}(\mu_{ic}, \sigma_{ic}^2)$. To maintain this distribution incrementally, each leaf node only stores three running scalar values per class-attribute pair: the instance count n , the sum of values $\sum x$, and the sum of squared values $\sum x^2$.

When a split evaluation occurs, these statistics are used to reconstruct the mean ($\mu = \frac{\sum x}{n}$) and variance ($\sigma^2 = \frac{\sum x^2}{n} - \mu^2$). The split-point candidates are then selected dynamically from an equidistant grid across the distribution’s range (typically bounded by $\mu \pm 3\sigma$).

- **Advantages:** Highly memory-efficient, requiring $O(1)$ constant space per attribute-class combination.
- **Disadvantages:** Struggles with multi-modal, highly skewed, or non-normal feature distributions, which can lead to sub-optimal split choices.

Histogram-based Approximation. The histogram approach relaxes any parametric assumptions regarding data distribution by maintaining an online, dynamic summary of the attribute’s values (e.g., via the Ben-Haim and Tom-Tov algorithm [3]). Each leaf tracks an array of dynamically updated bins that compress the empirical cumulative distribution function on the fly. When new values arrive, neighboring bins are merged if the total bin count exceeds a predefined threshold B .

- **Advantages:** Completely non-parametric and highly robust to non-normal, non-linear, or multi-modal data distributions, resulting in more accurate split-point selection.
- **Disadvantages:** Demands significantly more memory ($O(B)$ per attribute-class

pair) and incurs a higher computational overhead during bin-merging and split-point root-finding routines.

Hoeffding Tree Learning Procedure

The induction step for training the Hoeffding Tree proceeds as described in Algorithm 2.1:

- **Leaf Selection and Statistics Update (Lines 1–3):** For each incoming labeled instance (x, y) , the algorithm traverses the current tree T to locate the appropriate leaf node l (Line 1). It then updates the sufficient statistics at that leaf (Line 2) and increments the local instance counter n_l by 1 (Line 3).
- **Grace Period and Split Evaluation (Lines 4–7):** To reduce computational overhead, a split evaluation is only triggered every $n_{\min}$ instances, provided the leaf is not already pure (Line 4). When this condition is met, the split scores $G_l(X)$ (such as information gain) are computed for all features (Line 5), allowing the algorithm to identify the best feature X_a and the second-best feature X_b (Line 6). The difference between their split scores is then stored as ΔG (Line 7).
- **Hoeffding Bound Calculation (Line 8):** The Hoeffding bound ϵ is calculated based on the confidence parameter δ and the number of samples n_l seen at the leaf, establishing the statistical threshold required to confidently select a split attribute.
- **Tree Growth and Node Splitting (Lines 9–12):** If the best feature is valid ($X_a \neq X_\emptyset$) and the split metric difference ΔG exceeds ϵ —or if a tie occurs because ϵ drops below the threshold τ (Line 9)—the leaf l is officially split. The leaf is replaced by an internal decision node branching on X_a (Line 10), and a new, empty leaf node with initialized statistics is created for each resulting branch (Lines 11–12).

Algorithm 2.1 Hoeffding Tree Induction

Require: (x, y) : incoming labeled instance

Require: T : current tree

Require: δ : confidence parameter

Require: $n_{\min}$: grace period

Require: τ : tie-breaking threshold

- 1: Route (x, y) to a leaf l in T
 - 2: Update sufficient statistics at l
 - 3: $n_l \leftarrow n_l + 1$
 - 4: **if** $n_l \bmod n_{\min} = 0$ **and** l is not pure **then**
 - 5: Compute split scores $G_l(X)$ for all features X
 - 6: Let $X_a \leftarrow$ best feature, $X_b \leftarrow$ second-best feature
 - 7: $\Delta G \leftarrow G_l(X_a) - G_l(X_b)$
 - 8: Compute Hoeffding bound ϵ using δ and n_l
 - 9: **if** $X_a \neq X_b$ **and** ($\Delta G > \epsilon$ **or** $\epsilon < \tau$) **then**
 - 10: Replace leaf l with a decision node splitting on X_a
 - 11: **for** each branch of the split **do**
 - 12: Create a new leaf with initialized statistics
-

2.1.3. ADaptive WINdowing (ADWIN)

ADaptive WINdowing (ADWIN) [4] is an adaptive sliding window algorithm that monitors a stream of values (in this case, prediction errors) and automatically detects changes in their distribution.

ADWIN maintains a variable-length window of recent observations and continuously checks whether two subwindows exhibit statistically significant differences in their averages. When such a difference is detected, it indicates a change in the underlying distribution, triggering an error.

In this work, each tree is equipped with two instances of ADWIN, in particular:

- A **warning detector** (more sensitive) to signal a potential degradation in performance.
- A **drift detector** (more conservative) to confirm a significant change.

When a warning is detected, a background tree is initialized and trained in parallel on incoming data. If a drift is later confirmed, the current tree is replaced by the background one. This allows the model to react quickly and smoothly to changes while preserving

useful information when no drift occurs. In the rare case in which a drift is signaled before a warning, the tree is reset.

2.1.4. Adaptive Random Forest

Adaptive Random Forest (ARF) [19] is an ensemble framework specifically engineered for data stream classification. It adapts the traditional Random Forest paradigm to streaming constraints by utilizing an ensemble of M Hoeffding Trees [5] that grow incrementally.

ARF maintains high predictive accuracy and resilience to concept drift through three integrated mechanisms:

Online Bagging. Each tree is trained using online bagging. For every incoming instance (x, y) , a weight k is sampled from a Poisson distribution with parameter λ . The instance is then used to update the tree k times. This simulates bootstrap sampling [22] in an online setting and increases diversity among trees.

Random Feature Subspaces. To further decorrelate trees, each node evaluates splits using only a random subset of features. When a leaf is considered for splitting, a subset of features is sampled and stored locally in that node. All candidate splits at that node are restricted to this subset. This mechanism is analogous to the feature randomness in Random Forests, but applied incrementally during tree growth.

Drift Detection with ADWIN. Each tree is equipped with ADWIN, as better explained in Section 2.1.3, to ensure a highly responsive and precise drift adaptation of the model.

Prediction. Each tree produces a prediction independently. The final prediction is obtained by combining individual outputs, using different kind of voting mechanism, like majority voting, soft voting or a weighted vote based on tree performance.

Adaptive Random Forest Algorithm

The execution of the ARF algorithm proceeds as described in Algorithm 2.2:

- **Resampling and Prediction (Lines 1–4):** For each tree T_i in the ensemble, a weight k is sampled from a Poisson distribution parameterized by λ . The tree then makes a prediction $\hat{y}_i$ for the instance, and an error signal e_i is computed, which is equal to 1 if the prediction is incorrect and 0 otherwise.

- **Instance Skipping (Lines 5–6):** If the sampled Poisson weight k is 0, the current instance is skipped for this specific tree, and the algorithm proceeds to the next tree in the ensemble.
- **Model Training (Lines 7–9):** If $k > 0$, the main tree T_i is trained on the instance (x, y) with a training weight of k . Concurrently, if a background tree T_i^{bg} has already been initialized due to a previous concept warning, it is also trained on the same instance with weight k . Note that each leaf evaluates splits using its local feature subspaces.
- **Concept Drift and Warning Detection (Lines 10–18):** The error signal e_i is generated and used to update both the warning and drift detectors.
 - If a *warning* status is triggered (Lines 12–13), a new background tree T_i^{bg} is initialized in anticipation of a potential concept drift.
 - If a *drift* status is triggered (Lines 14–18), the current tree T_i is replaced by the background tree T_i^{bg} if it exists; otherwise, T_i is reset completely to adapt to the new data distribution.
- **Ensemble Voting (Line 19):** Finally, after iterating through all trees, individual predictions $\{\hat{y}_i\}$ are aggregated (e.g., via a majority or weighted vote) to yield the final ensemble output.

Algorithm 2.2 Adaptive Random Forest

Require: (x, y) : incoming labeled instance**Require:** $\{T_i\}_{i=1}^M$: ensemble of trees**Require:** λ : bagging parameter

```

1: for each tree  $T_i$  in the ensemble do
2:   Sample  $k \sim \text{Poisson}(\lambda)$ 
3:   Predict  $\hat{y}_i$  from  $T_i$ 
4:   Compute error signal  $e_i = \mathbb{I}(\hat{y}_i \neq y)$ 
5:   if  $k = 0$  then
6:     continue
7:   Train  $T_i$  on  $(x, y)$  with weight  $k$            ▷ Each leaf evaluates splits using its local
   feature subset
8:   if background tree  $T_i^{bg}$  exists then
9:     Train  $T_i^{bg}$  on  $(x, y)$  with weight  $k$        ▷ Each leaf evaluates splits using its local
   feature subset
10:  Update warning detector with  $e_i$ 
11:  Update drift detector with  $e_i$ 
12:  if warning detected then
13:    Initialize background tree  $T_i^{bg}$ 
14:  if drift detected then
15:    if background tree exists then
16:      Replace  $T_i \leftarrow T_i^{bg}$ 
17:    else
18:      Reset  $T_i$ 
19: Combine predictions  $\{\hat{y}_i\}$  (e.g., weighted vote)

```

2.1.5. Streaming Random Patches

Streaming Random Patches (SRP) [20] is an ensemble method for data stream classification designed to improve predictive performance and robustness in the presence of concept drift. It shares several components with Adaptive Random Forest (ARF) described at 2.1.4, such as online bagging and drift detection, but differs fundamentally in how randomness is introduced.

Global Feature Subspaces. The key characteristic of SRP is the use of *global feature subspaces*. Each base learner is trained on a fixed subset of features, sampled once at initialization and kept constant throughout its lifetime. This contrasts with ARF, where

feature randomness is applied locally at each node.

As a result, each model learns from a different *projection* of the input space, increasing diversity at the ensemble level. This approach is inspired by the Random Subspaces and Random Patches methods, where models are trained on subsets of both instances and features.

Online Bagging. Similarly to ARF, SRP uses online bagging to further increase diversity. For each incoming instance (x, y) , a weight k is sampled from a Poisson distribution, and the instance is used to update each learner k times. This simulates bootstrap sampling in a streaming setting.

Drift Detection with ADWIN. Each tree is equipped with ADWIN, as better explained in Section 2.1.3, to ensure a highly responsive and precise drift adaptation of the model.

Training Variants. SRP generalizes multiple ensemble strategies depending on how subsets are sampled:

- **SRP:** subsets of both instances and features (default configuration);
- **Random Subspaces (SRS):** subsets of features only;
- **Bagging:** subsets of instances only.

This flexibility allows SRP to balance computational cost and predictive performance.

Prediction. Each learner produces a prediction using only its feature subset. The final prediction is obtained by combining outputs across the ensemble, typically using majority or weighted voting.

Streaming Random Patches Algorithm

The execution of the SRP algorithm is detailed in Algorithm 2.3. The streaming process operates through three main phases per incoming instance:

- **Projection and Resampling (Lines 1–7):** The incoming instance x is immediately restricted to the learner’s pre-allocated global feature subset F_i . Online bagging is then simulated via a Poisson distribution to determine the instance weight k .

- **Adaptive Training (Lines 8–10):** If $k > 0$, both the primary learner T_i and any existing background learner T_i^{bg} (instantiated during a prior warning state) are trained on the projected instance.
- **Drift Detection and Voting (Lines 11–21):** Prediction errors update the ADWIN warning and drift detectors. A warning triggers the initialization of a background learner with a fresh feature subspace, while a confirmed drift replaces the main model. Finally, individual subspace predictions are aggregated.

Algorithm 2.3 Streaming Random Patches

Require: (x, y) : incoming labeled instance

Require: $\{T_i\}_{i=1}^M$: ensemble of learners

Require: λ : bagging parameter

Require: m : feature patch size

```

1: for each learner  $T_i$  in the ensemble do
2:    $x_i \leftarrow x$  restricted to features in  $F_i$   $\triangleright$  Each learner has a fixed feature subset  $F_i$  of
      size  $m$ 
3:   Sample  $k \sim \text{Poisson}(\lambda)$ 
4:   Predict  $\hat{y}_i$  using  $T_i$  on  $x_i$ 
5:   Compute error signal  $e_i = \mathbb{I}(\hat{y}_i \neq y)$ 
6:   if  $k = 0$  then
7:     continue
8:   Train  $T_i$  on  $(x_i, y)$  and on  $F_i$  with weight  $k$ 
9:   if background learner  $T_i^{bg}$  exists then
10:    Train  $T_i^{bg}$  on  $(x_i, y)$  and on  $F_i$  with weight  $k$ 
11:   Update warning detector with  $e_i$ 
12:   Update drift detector with  $e_i$ 
13:   if warning detected then
14:     Initialize background learner  $T_i^{bg}$  with a new feature subset
15:   if drift detected then
16:     if background learner exists then
17:       Replace  $T_i \leftarrow T_i^{bg}$ 
18:     else
19:       Reset  $T_i$  with a new feature subset
20: Combine predictions  $\{\hat{y}_i\}$  (e.g., weighted vote)

```

SRP vs ARF

While ARF and SRP share structural foundations—namely online bagging and AD-WIN-based drift management—their core divergence lies in the architectural placement of randomness, resulting in distinct performance trade-offs.

Feature Randomization Strategy. The primary distinction belongs to the *granularity of feature randomization*, as illustrated in Figure 2.1:

- ARF applies **local feature subspaces** dynamically at each node split, enabling fine-grained adaptation within a single tree.
- SRP enforces **global feature subspaces** fixed at initialization per learner, forcing models to specialize in entirely different projections of the input space.

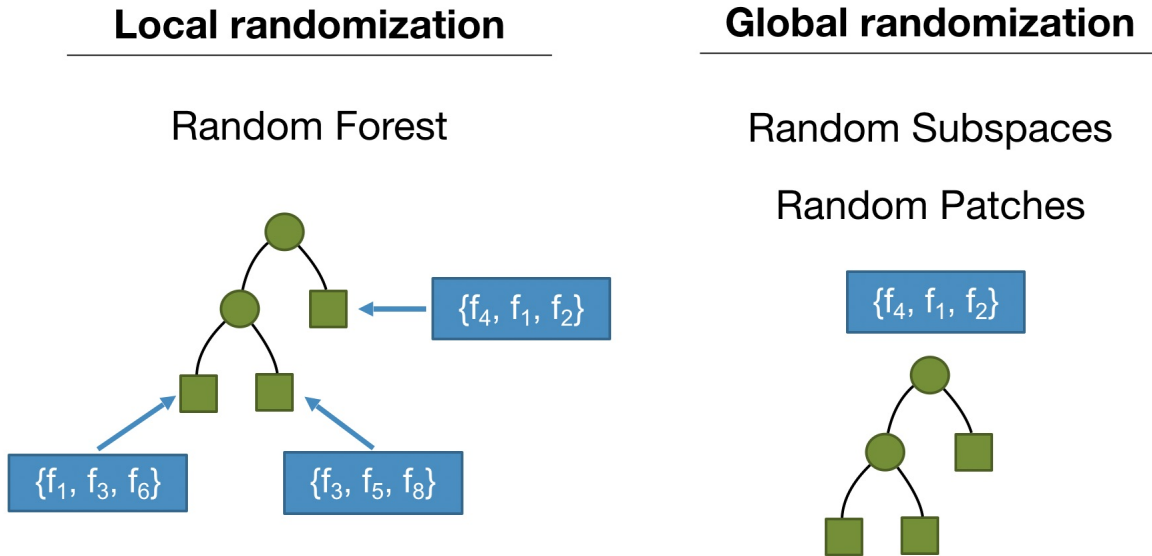


Figure 2.1: Local and global feature subset randomization

Model Flexibility and Base Learners. While ARF is algorithmically bound to Hoeffding Trees due to its node-level mechanics, SRP is entirely model-agnostic. It can leverage any incremental learning algorithm as a base learner, providing greater architectural flexibility.

Diversity, Memory, and Performance Trade-offs. These structural choices dictate their real-world behavior:

- **Diversity Type:** ARF promotes *intra-model diversity* (complex, expressive individual trees), whereas SRP fosters *inter-model diversity* (ensemble-level robust

	Adaptive Random Forest	Streaming Random Patches
Feature subspace	Local (per node)	Global (per model)
Base learner	Hoeffding Tree	Any incremental learner
Diversity type	Intra-model	Inter-model
Memory efficiency	Higher	Lower

Table 2.1: Comparison of ARF and SRP characteristics

specialization).

- **Resource Consumption:** ARF is generally more memory-efficient as trees share the full feature space footprint. SRP can incur overhead from managing independent data projections.
- **Predictive Accuracy:** Though both scale exceptionally well with ensemble size, SRP typically edges out ARF in high-dimensional settings with complex feature interactions due to its subspace specialization.

Table 2.1 summarizes these fundamental differences.

2.1.6. Mondrian Tree

In contrast to the Hoeffding Tree, which relies on incremental statistical updates for split decisions, the Mondrian Tree adopts a non-parametric approach rooted in **computational geometry**. It partitions the data space through a series of hierarchical boundaries while maintaining robust theoretical guarantees regarding consistency and convergence [31].

Spatial Partitioning and Hyperrectangles. Under this geometric paradigm, each data instance $x \in \mathcal{X}$ is represented as a point within a d -dimensional feature space. The domain of a node is defined by a **hyperrectangle** B , bounded by the coordinate-wise minima and maxima of the observed feature values:

$$B = \prod_{j=1}^d [a_j, b_j]$$

where a_j and b_j denote the lower and upper bounds of the j -th feature, respectively.

The structure is constructed via **guillotine partitioning**, a recursive process of applying axis-aligned cuts that splits a parent hyperrectangle into two disjoint child hyperrectangles. Each split is defined by an axis j and a threshold τ , such that data points

are routed based on the condition $x_j \leq \tau$. As illustrated in Figure 2.2, a single cut along the first axis ($j = 1$) with a threshold of $\tau = 0.5$ that divides the parent domain into two distinct, non-overlapping spatial regions.

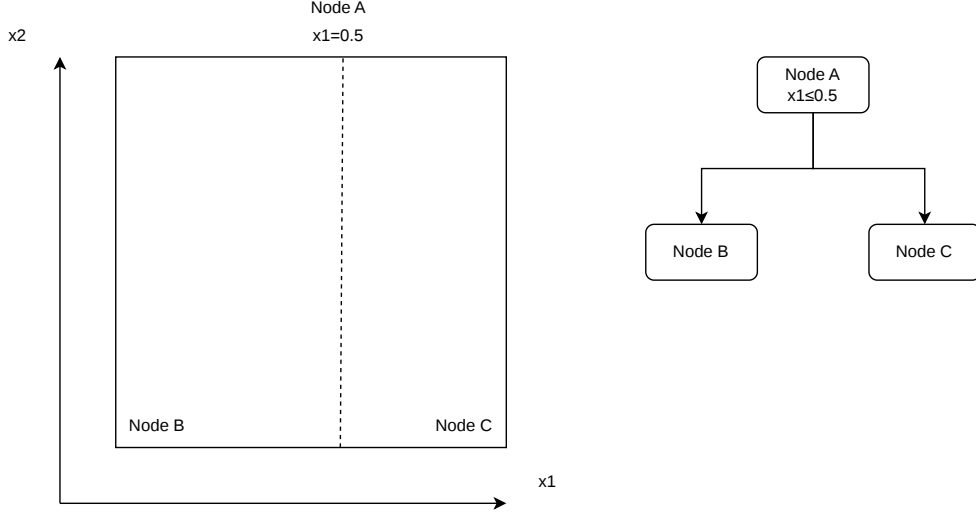


Figure 2.2: Left: A single guillotine cut in the 2D feature space. Right: The corresponding tree structure.

Restricted Mondrian Process. The **Restricted Mondrian Process** [32] governs the dynamic evolution of the tree structure in a streaming context. A critical distinction between the restricted and Unrestricted Mondrian Processes lies in the handling of empty space. The unrestricted process samples splits across the entire theoretical domain, often generating empty leaves that contain no data points. Conversely, the restricted process confines its splits strictly to the empirical range of the observed features. This constraint ensures that no empty leaves are generated.

Unlike traditional decision trees that split only when a statistical criterion (e.g., Gini impurity or Information Gain) is met, the Mondrian Process treats the split occurrence as a stochastic event. It operates entirely independent of the class labels, basing split decisions and intensities solely on spatial distances and geometry. As a new sample (x, y) arrives, Algorithm 2.4 updates the tree through the following sequence:

- **Range Extension:** When a new point x falls outside a node's current bounding box, the algorithm computes the total linear extension required to include it, which directly determines the split rate λ .
- **Temporal Split Condition:** A candidate split time is sampled from $\text{Exponential}(\lambda)$. A split is triggered if the node is a leaf, or if the candidate time precedes the creation

time of the existing children. If neither condition holds, the sample is routed deeper and the node's bounding box is extended to include $\mathbf{x}$.

- **Threshold Selection:** If a split is triggered, the split axis j is chosen proportionally to its spatial extension, and the threshold is drawn uniformly from within that interval. The old subtree is preserved in one child, and the new sample is placed in the other.

Algorithm 2.4 Restricted Mondrian Process Update

Require: Node u , Sample (x, y)

- 1: Range extension $\lambda \leftarrow \sum_j (\max(x_j - u.b_j, 0) + \max(u.a_j - x_j, 0))$ $\triangleright$ Range Extension
 - 2: Sample time step $\Delta t \sim \text{Exponential}(\lambda)$
 - 3: $t_{\text{split}} \leftarrow u.\text{time} + \Delta t$
 - 4: **if** u is a leaf **or** $t_{\text{split}} < u.\text{child.time}$ **then** $\triangleright$ Temporal Split Condition
 - 5: Split at threshold π on a random feature j $\triangleright$ Threshold Selection
 - 6: Generate two new child nodes for u
 - 7: One child receives the old box structure, the other receives the new sample (x, y)
 - 8: Update bounding box bounds $u.a$ and $u.b$ to include $\mathbf{x}$
-

Bayesian Probability Estimation. To estimate class probabilities $P(c, x)$, the Mondrian Tree employs a **Dirichlet Process** [34]. This serves as a Bayesian smoothing technique to handle the "zero-frequency" problem in sparse data streams. The probability for class c is given by:

$$P(c, x) = \frac{n_c + \alpha}{N + K\alpha} \quad (2.2)$$

where n_c is the count of class c , N is the total samples in the node, K is the cardinality of the label space, and α is the **Dirichlet prior**. A larger α regularizes the model toward a uniform distribution, providing stability during the initial stages of the stream.

Hierarchical Weighting via Context Tree Weighting (CTW). During inference, the model leverages CTW [37] to aggregate predictions across the hierarchy. Each node v maintains a local weight W_v reflecting its historical accuracy, alongside an aggregated weight L_v representing the overall performance of its subtree. The final prediction is a mixture of the node's local estimate P_v and the smoothed estimate of its descendants P_{desc} :

$$P_{final}(v) = \frac{1}{2}\eta_v P_v + \left(1 - \frac{1}{2}\eta_v\right) P_{desc} \quad (2.3)$$

where η_v is a dynamic weighting factor derived from the log-difference between the local node and its subtree, computed as $\eta_v = \exp(W_v - L_v)$.

Loss Optimization and Adaptability. The node weights are updated using a **Logarithmic Loss** [10] function. Upon observing the true label y , the node suffers a penalty:

$$Loss = -\ln(P(y)) \quad (2.4)$$

The weight W_v is updated as:

$$W_{v,\text{new}} = W_{v,\text{old}} - \text{step} \cdot \text{Loss} \quad (2.5)$$

where step represents the learning rate. This mechanism ensures the ensemble can adapt to **concept drift** by rapidly de-weighting nodes whose geometric partitions no longer accurately reflect the underlying data distribution.

2.1.7. Aggregated Mondrian Forest

Aggregated Mondrian Forest (AMF) is an online random forest algorithm for data stream classification driven by the continuous spatial partitioning of the Restricted Mondrian process. While it typically produces predictions through an exact, computationally efficient aggregation of all possible subtrees, a non-aggregated variant of the algorithm can also be used.

Prediction

The prediction phase (Algorithm 2.5) in AMF relies on a bottom-up aggregation process repeated across all M trees in the ensemble. When a new instance requires a prediction, it is first routed down each tree to its corresponding leaf node, where an initial class probability estimate is obtained. From this leaf, the algorithm performs an upward recursion back to the root, applying Context Tree Weighting (CTW) at each step. CTW combines the node's local prediction P_v with the already-aggregated prediction from below, weighted by η_v — a dynamic factor reflecting how much to trust the node's individual estimate versus its subtree. The final forest prediction is the average of all tree outputs, yielding a probability distribution over all possible classes.

Algorithm 2.5 Aggregated Mondrian Forest (AMF) Prediction

Require: x : incoming instance to predict

Require: $\{T_i\}_{i=1}^M$: ensemble of M Mondrian trees

```

1:  $P_{\text{total}} \leftarrow 0$ 
2: for each tree  $T_i \in \{T_i\}_{i=1}^M$  do
3:   Route  $x$  to a leaf  $v$  in  $T_i$ 
4:    $P_{\text{agg}} \leftarrow P_v$  ▷ Initial prediction at leaf node  $v$ 
5:    $v \leftarrow \text{parent}(v)$ 
6:   while  $v \neq \text{null}$  do ▷ Upward Aggregation (CTW)
7:      $\eta_v \leftarrow \exp(W_v - L_v)$ 
8:      $P_{\text{agg}} \leftarrow \frac{1}{2}\eta_v P_v + (1 - \frac{1}{2}\eta_v) P_{\text{agg}}$ 
9:      $v \leftarrow \text{parent}(v)$ 
10:   $P_{\text{total}} \leftarrow P_{\text{total}} + P_{\text{agg}}$  ▷ Accumulate tree predictions
11: return  $\frac{1}{M}P_{\text{total}}$  ▷ Final forest prediction via soft weighting

```

Train

The learning phase (Algorithm 2.6) in AMF relies on a continuous top-down expansion and bottom-up update process, repeated for each tree in the ensemble. When a new labeled instance (x, y) arrives, it is routed down the tree while the Restricted Mondrian Process (Algorithm 2.4) is applied at each node, potentially triggering at most one split per incoming sample. At every internal node visited, the local weight W_v is penalized using the log-loss incurred at that node. If a split is triggered, the new sample is routed into a newly created leaf, while the pre-existing subtree remains intact in the other branch.

Once the sample reaches a terminal leaf, the bottom-up update begins. For a leaf node, its aggregated subtree weight L_v is set equal to its local weight W_v , reflecting that it has no descendants. For each internal node encountered during the upward recursion, the aggregated weight L_v is recomputed via a log-sum-exp combination of the node's own local weight and the combined weights of its two children $L_c = L_{\text{left}} + L_{\text{right}}$. This step implements the CTW rebalancing mechanism, dynamically adjusting each node's trust between its own local estimate and its subtree's collective performance.

Algorithm 2.6 Aggregated Mondrian Forest (AMF) Train

Require: (x, y) : incoming labeled instance

Require: $\{T_i\}_{i=1}^M$: ensemble of Mondrian trees

```

1: for each tree  $T_i \in \{T_i\}_{i=1}^M$  do
2:    $v \leftarrow \text{root}(T_i)$ 
3:   while  $v$  is not a leaf do                                ▷ Top-down traversal and tree extension
4:     Evaluate Restricted Mondrian Process on  $v$  with  $x$ 
5:     Update local weight  $W_v$  using loss function
6:     if split occurred then
7:        $v \leftarrow$  newly created leaf containing  $x$ 
8:       break
9:     else
10:       $v \leftarrow$  child of  $v$  containing  $x$ 
11:      Update bounding box of  $v$  to include  $x$ 
12:   Update bounding box of leaf  $v$  to include  $x$ 
13:   while  $v \neq \text{null}$  do                                    ▷ Bottom up Update
14:     if  $v$  is a leaf then
15:        $W_v \leftarrow L_v$ 
16:     else
17:        $L_c \leftarrow L_{v.\text{left}} + L_{v.\text{right}}$ 
18:        $W_v \leftarrow \ln\left(\frac{1}{2} \exp(W_v) + \frac{1}{2} \exp(L_c)\right)$ 
19:    $v \leftarrow v.\text{parent}$ 

```

2.2. The Renoir Dataflow Runtime

This section introduces the Renoir platform, focusing only on the components relevant to this thesis. While Renoir supports a wide range of functionalities, including batch processing and advanced analytics, we restrict the discussion to the elements used in our distributed streaming learning implementation.

2.2.1. Overview and Goals

REactive Network of Operators In Rust (Renoir) [12] is a distributed data processing framework written in Rust, designed to provide a high-level programming model while maintaining low runtime overhead. Its core abstraction is the *dataflow model*, where a computation is expressed as a directed graph of operators. Each operator applies a

transformation to incoming data and produces output that is consumed by downstream operators.

This model provides a structured way to express complex processing pipelines. By making data dependencies explicit, it naturally enables parallel execution: independent operators can run concurrently, and multiple instances of the same operator can process different partitions of the data in parallel. At the same time, the abstraction simplifies development by delegating low-level concerns such as scheduling, communication, and synchronization to the runtime system.

A key design goal of Renoir is to combine this high-level abstraction with efficient execution. In streaming scenarios, where data arrives continuously and must be processed at high throughput, minimizing overhead is essential. Unlike many existing frameworks built on the Java Virtual Machine (JVM), Renoir leverages Rust as a systems programming language to achieve better performance. In particular, it benefits from static dispatch, compile-time optimizations, and memory safety without garbage collection. Through monomorphization, user-defined operators are compiled into specialized machine code, reducing abstraction overhead and improving execution efficiency.

From an execution standpoint, Renoir targets distributed environments composed of multiple hosts. Each host runs a worker process, which internally exploits multi-threading to achieve intra-node parallelism. A dataflow program is translated into a logical execution plan and deployed across these workers, enabling scalable distributed computation.

2.2.2. Programming Model

Renoir adopts a dataflow programming model that represents computations as transformations applied to data streams. This model separates the logical definition of a computation from its physical execution, allowing the runtime system to optimize parallelism and communication.

The API is built around four main abstractions:

- **Streams.** The central abstraction is `Stream<T>`, representing a (possibly unbounded) sequence of elements of type T . Streams are typed and form the backbone of the dataflow graph. To support distributed execution, the type T must be serializable.
- **Sources.** Sources introduce data into the system. They are operators with no inputs that generate streams from external data, such as files, network inputs, or in-memory collections. In practice, iterator-based sources are commonly used to

create streams from sequential data.

- **Operators.** Operators are the fundamental processing units. They consume one or more input streams, apply transformations, and produce output streams. Transformations are typically expressed in a functional style, enabling concise and composable data processing pipelines.
- **Sinks.** Sinks terminate the dataflow by consuming streams and producing outputs outside the system, such as writing results to memory, storage, or external services.

This abstraction enables a clear separation between *what* is computed and *how* it is executed. The programmer defines the computation as a graph of operators, while the runtime system handles execution details such as data partitioning, scheduling, and inter-node communication.

As a result, Renoir can efficiently exploit both parallelism and distribution: multiple instances of an operator can process different portions of a stream concurrently, and independent operators can execute in parallel as soon as their inputs are available.

2.2.3. Operators

Operators are the basic computational units in the Renoir dataflow model. Each operator consumes one or more input streams, applies a transformation, and produces output streams. By composing operators, complex streaming pipelines can be defined as a logical dataflow graph executed by the runtime.

A fundamental distinction is between *stateless* and *stateful* operators. Stateless operators compute outputs solely from the current input element, without retaining information across events, ensuring deterministic behavior. In contrast, stateful operators maintain and update internal state over time, making outputs dependent on previously observed data. This requires careful coordination by the runtime, typically through stream partitioning, so that all elements affecting a given state are processed by the same operator instance. This work uses a subset of Renoir operators sufficient to express the target pipeline.

Stateless operators

- **map:** applies an element-wise transformation to each input item;
- **flat_map:** produces zero, one, or multiple output elements from a single input, enabling event expansion;

- `filter_map`: combines filtering and mapping by emitting only values wrapped in `Some`, discarding others;
- `drop_key`: forgets about the key of the stream and returns a stream containing just the values.

Stateful operators Stateful operators encapsulate the core logic:

- `rich_map`: a stateful variant of `map` that maintains partition-local state, enabling incremental computations;
- `rich_map_transient`: extends `rich_map` with explicit state removal when it is no longer needed.

Overall, Renoir provides a unified abstraction for combining stateless and stateful transformations, enabling modular and scalable streaming computations.

2.2.4. Parallel Execution Model

Renoir translates the logical dataflow graph into an efficient execution plan by exploiting both *task parallelism* and *data parallelism*. The former allows different operators to run concurrently, while the latter enables multiple instances of the same operator to process disjoint partitions of the input stream.

Stream Partitioning Parallel execution relies on partitioning streams so that multiple operator instances can process data independently. Renoir provides:

- `group_by`: partitions data by key, ensuring that elements with the same key are routed to the same instance. This is essential for maintaining consistent partitioned state;
- `repartition_by`: allows changing the replication and forwarding strategy, for example to force an operator to execute as a single instance on a single designated host.

2.2.5. Sinks

While Renoir provides different sinks to collect the output, we chose to stick to `write_csv`, a simple operator that writes the final output of the dataflow to a CSV file, for later analysis.

2.3. Motivation

The preceding sections have laid out two foundational pillars of this work: (i) the landscape of SML with some of its learners, and (ii) the Renoir distributed dataflow runtime.

The central motivation of this thesis is to demonstrate that these two pillars can be systematically combined to create a new class of distributed, high-throughput and high-accuracy ensemble learners. Although the work on Vertical Hoeffding Tree (VHT) [11, 24] successfully demonstrated this feasibility for a single, vertically-partitioned Hoeffding Tree, the challenges and opportunities presented by ensembles are fundamentally different.

Ensemble methods like Streaming Random Patches (SRP), Adaptive Random Forest (ARF) and Aggregated Mondrian Forest (AMF) represent the current frontier of predictive performance in SML, aggregating multiple, diverse base models. This inherent parallelism makes them natural candidates for distribution. However, the very mechanisms that make them accurate—i.e. diversity injection (via bootstrapping and subspaces), adaptive drift detection, and collective voting—introduce a new class of coordination and state management problems that are not present in a single-learner scenario.

This problem requires a principled re-architecture to answer questions such as:

- How can we maintain the statistical integrity of ensemble diversity when instances are processed in parallel across distributed workers?
- How can we implement consensus-based voting mechanisms without creating a single, serializing bottleneck?
- How can we manage the lifecycle of individual trees, including their resetting upon drift detection, in a way that is consistent with the continuous, asynchronous nature of a dataflow graph?
- How do the overheads of coordination for an ensemble compare to the coordination overheads of a complex single model like the VHT?

This thesis argues that Renoir’s specific blend of features makes it uniquely suited, and perhaps optimal, for addressing these challenges. Its position as a high-performance Rust-based runtime offers a critical advantage over the incumbent platforms in this space. Known frameworks like Massive Online Analysis (MOA) and Scalable Advanced Massive Online Analysis (SAMOA), while providing robust algorithmic foundations, are built on the JVM. This introduces significant runtime overheads (garbage collection, compilation variability, less control over memory layout) that can become severe bottlenecks when

scaling to the throughputs required by modern data streams. Similarly, popular Python-based alternatives such as River [29], despite offering highly accessible ecosystems, are fundamentally constrained by Python’s design, limited by single-threaded execution. This creates a severe bottleneck that explicitly limits the throughput, performance, and scalability of computationally demanding techniques like ensemble methods.

Therefore, our work is motivated by four core objectives:

- **To Formulate a Scalable Dataflow Architecture for SML Ensembles.** We aim to map the computational patterns of SRP, ARF, and AMF onto Renoir’s operator graph. This involves decomposing ensemble learning into a pipeline of data-parallel and task-parallel stages, from instance ingestion and broadcast, to parallel tree maintenance, to vote aggregation and model update coordination.
- **To Design an Agnostic and Extensible Framework.** We aim to develop a unified architecture that remains agnostic to specific ensemble methods. By decoupling the streaming dataflow logic from the underlying learning algorithms, we aim to create a system where adding a new ensemble method is trivial, requiring only the implementation of specific operator logic without reconfiguring the entire distributed pipeline.
- **To Leverage Renoir’s Performance for High-Throughput Ensemble Learning.** We will exploit Renoir’s lightweight workload, and fine-grained control over operator parallelism to build a system that can ingest and learn from streams at rates that are unattainable on centralized platforms like MOA.
- **To Systematically Analyze the Throughput-Accuracy Trade-offs in Distributed Ensembles.** Beyond simply building a faster system, we will conduct an empirical study. We will investigate how design choices—such as the degree of tree-level parallelism, batching strategies, and the overhead of drift detection—affect both the throughput and the final predictive accuracy of the models. Crucially, we will compare our horizontal ensemble approach not only against centralized baselines but also against the vertical single-tree approach cited before.

By achieving these objectives, this thesis will demonstrate that the dataflow paradigm, and Renoir in particular, is not just a viable but an optimal platform for building a high-performance, accurate, and robust streaming machine learning systems, showing that the complexity of ensemble coordination can be mastered within a simple, expressive, and extremely fast runtime like Renoir.

3 | Design and Implementation

This chapter presents the design and implementation of a distributed streaming ensemble learning system built on the Renoir dataflow runtime. The architecture balances abstraction, efficiency and modularity: a trait-based design enables multiple ensemble methods to coexist in a single codebase, while careful data stream management via key-based routing eliminates the need for a centralized master node.

The chapter is organized as follows. Section 3.1 details the core abstractions and data structures underpinning the system. Section 3.2 describes how instances flow through the distributed pipeline, from input to final prediction.

3.1. Software Architecture

The system is implemented in Rust, a choice motivated by the requirement for high-throughput performance in streaming contexts and native compatibility with the Renoir dataflow runtime. The architecture is designed not merely as a fixed implementation of specific algorithms, but as a **generalized framework** for online ensemble learning. It prioritizes three primary design objectives: (i) the support of diverse ensemble methods within a unified, extensible codebase, (ii) the enablement of stateful, per-partition processing within the dataflow, and (iii) the minimization of memory overhead through the use of cache-efficient data structures.

These objectives are realized through a decoupled, multi-layer design: an **abstraction layer** that defines the learner lifecycle, an **intermediate generalization layer** that harvests commonalities between algorithmic families, and an **implementation layer** where specific logic is injected.

3.1.1. Core Abstraction: Online Learner

At the heart of the design is the `OnlineLearnerTrait`, which defines a standard interface for all base learners. This abstraction ensures that the streaming pipeline remains entirely agnostic to the underlying learning algorithm. By treating learners as modular

components, the architecture enables the seamless integration of new algorithms without requiring modifications to the core dataflow logic.

The hierarchical relationship between the core abstraction and the concrete implementations is illustrated in Figure 3.1. A key innovation is the introduction of `HTBasedLearner` as an additional abstraction layer. Rather than implementing SRP and ARF as isolated entities, `HTBasedLearner` acts as a **generalization middle layer**. Since these algorithms share a common foundation in Hoeffding Tree logic, this layer encapsulates their shared behaviors—such as drift detection and attribute observers—while delegating specific structural nuances to specialized sub-components. This reduces code duplication and ensures that any improvement to the base Hoeffding-based ensembles logic benefits all derived ensemble methods.

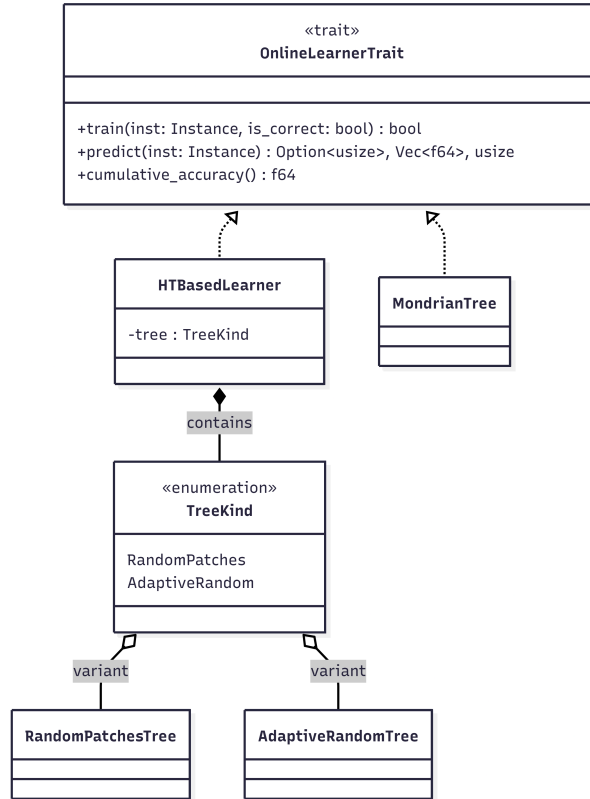


Figure 3.1: High-level class diagram of the learner abstraction.

The trait exposes three fundamental methods that define the learner’s lifecycle:

- **predict**: Computes a prediction and a corresponding vote vector for a given input instance.
- **train**: Updates the internal model state using a labeled instance, adhering to the *test-then-train* evaluation paradigm.

- `cumulative_accuracy`: Returns a running estimate of the learner’s predictive performance, serving as the basis for dynamic weighted voting schemes.

This design enforces a **strict separation of concerns**: the dataflow pipeline invokes these methods as black-box operations through the use of the `OnlineLearner` enum. This makes the architecture highly extensible; the three ensemble methods currently implemented serve as a proof-of-concept for how easily the system can be scaled to support dozens of different learning strategies.

3.1.2. Configuration and Parameterization

The system architecture employs a highly flexible, declarative configuration model based on JSON. During the initialization phase, a `config.json` file is deserialized into a strictly-typed internal representation. A major strength of this architecture is its **dependency injection capability**: the configuration object is captured by the dataflow closures, allowing hyperparameters to be easily adjusted and injected into the processing nodes without recompiling the core logic.

The configuration schema is partitioned into global orchestration settings and algorithm-specific hyperparameters. The latter uses a tagged union pattern (polymorphism), where the `type` field determines the specific parameters required for the chosen learner.

Global Ensemble Settings

The top-level JSON object contains the parameters described in Table 3.1. These fields govern the high-level orchestration of the Renoir dataflow and the final prediction aggregation logic, independent of the underlying learner type.

Parameter	Type	Description
<code>dataset</code>	String	Path or identifier of the input data stream.
<code>n_trees</code>	Integer	Total number of base learners (<code>OnlineLearnerTrait</code>).
<code>voting</code>	Enum	Aggregation strategy: <code>Majority</code> , <code>Soft</code> , or <code>Weighted</code> .
<code>seed</code>	Int	Starting seed used by the trees
<code>algorithm</code>	Object	Tagged union containing the specific learner parameters.

Table 3.1: Global configuration parameters.

Algorithm-Specific Hyperparameters

The internal `algorithm` object varies significantly depending on whether a Hoeffding-based (HB, i.e. SRP, ARF) or a Mondrian-based (AMF) model is selected. Table 3.2 provides a comprehensive map of all available hyperparameters across these implementa-

tions.

Category	Parameter	Learner	Description
General	type	All	"srp", "arf", or "amf".
Structure	n_min	HB	Grace period before split attempts.
	max_bins	HB	Max bins for histogram estimators.
	numeric_estimator	HB	"Gaussian" or "Histogram".
Split Logic	delta	HB	Hoeffding bound confidence $(1 - \delta)$.
	tau	HB	Tie-breaking threshold.
Ensemble	features_patch	HB	Percentage of features in subspace $([0, 1])$.
	lambda	HB	Poisson parameter for online bagging.
Drift	drift_detection	HB	Boolean flag to enable ADWIN monitoring.
	adwin_delta_warning	HB	Sensitivity for warning state.
	adwin_delta_drift	HB	Sensitivity for model reset.
Mondrian	step	AMF	Learning rate for weight updates.
	dirichlet	AMF	Concentration parameter for class priors.
	split_pure	AMF	Permits splitting of class-pure nodes.

Table 3.2: Algorithm-specific configuration hyperparameters.

3.1.3. Implementation Details and State Management

While the higher-level dataflow pipeline interacts solely with abstract streaming components, the underlying framework must maintain, mutate, and isolate complex model states across parallel processing nodes. The system utilizes Rust’s strict memory safety guarantees and zero-cost abstractions to implement a state management model that optimizes data-locality and prevents data races without relying on expensive synchronization primitives like mutexes.

The fundamental unit of data traversing the stream is the **Instance** structure. This structure encapsulates a dense feature vector and an optional categorical label. Upon ingestion, each instance is stamped with a monotonic, sequential **instance_id**, enabling downstream nodes to audit pipeline latencies and track evaluation windows.

High-Performance Polymorphism: Enums vs. Trait Objects

Architecting a flexible, multi-model ensemble system typically requires polymorphism. Traditional object-oriented patterns rely on runtime polymorphism via dynamic dispatch (e.g., vtables). In streaming applications characterized by high arrival rates, dynamic dispatch introduces instruction cache misses and prevents the compiler from performing aggressive function inlining.

To circumvent this performance bottleneck, the framework leverages Rust’s **Algebraic Data Types (ADTs)** via the **TreeKind** and **OnlineLearner** enumerations. Rather than

passing pointers to heap-allocated trait objects (`Box<dyn OnlineLearnerTrait>`), the dataflow workers use static dispatch managed through pattern matching on these enums. This layout offers two primary system benefits:

- **Contiguous Memory Allocation:** Enums are sized at compile-time to match their largest variant. This ensures that arrays or vectors of learners are allocated contiguously in memory, significantly maximizing CPU L1/L2 cache locality during rapid sequential model evaluations.
- **Dataflow Serialization Boundary:** Because Renoir must serialize and ship states across parallel task partitions, objects must implement the `Serialize` and `Deserialize` traits. Enums seamlessly support derive-macros for structural serialization, whereas dynamic trait objects cannot be easily serialized out-of-the-box.

The Middle Layer: `HTBasedLearner` State Machine

The `HTBasedLearner` serves as a stateful, intermediate manager that encapsulates the lifecycle of a single ensemble slot. It acts as an orchestrator that combines a primary decision tree, a dynamic Poisson distribution sampler for online bagging, and a dual-state concept drift monitor.

Stateful Online Bagging. The system implements a non-blocking streaming version of online bagging. Instead of copying historical data instances, the learner estimates sample repetition weights using a Poisson distribution. For each arriving instance, the `HTBasedLearner` lazily instantiates a thread-local, pseudo-random number generator (`SmallRng`), seeds it deterministically via a configured base seed, and samples an integer weight $k \sim \text{Poisson}(\lambda)$. If $k > 0$, the underlying tree variant is trained iteratively k times using the instance, preserving the statistical property of standard bootstrap bagging without memory overhead.

Drift-Driven State Transitions. Concept drift adaptation is governed by an instance of a `DualAdwin` detector. The state transitions are explicitly decoupled from the outer dataflow framework and operate inside the learner’s `train` loop, transitioning as follows:

1. **Normal Execution (Stable State):** The learner receives an instance, updates its running cumulative accuracy, computes the bagging weight k , and trains the primary tree variant (`RandomPatchesTree` or `AdaptiveRandomTree`). The model tracking error (0.0 for correct predictions, 1.0 for incorrect) is pushed to the drift detector.

2. **Warning State Escalation:** When the drift detector signals a statistical variation exceeding the warning threshold, the `HTBasedLearner` triggers the background learner allocation. It invokes a localized factory function (`new_tree()`) to spawn a completely fresh, isolated companion tree variant. Crucially, this variant is assigned its own random feature subspace partition (for SRP) or an independent random seed (for ARF). From this point forward, incoming streaming items are fed simultaneously to both the primary tree and the background candidate tree, allowing the candidate to warm up its sufficient statistics in parallel.
3. **Drift State and Model Replacement:** If the drift monitor's window statistics cross the severe drift threshold, a model swap occurs. The old, degraded primary tree is dropped from memory, and the pre-warmed background learner is instantly promoted to the primary slot via an in-place reference assignment. The running evaluation metrics (`cumulative_correct` and `cumulative_n`) are reset to zero to prevent historical bias from contaminating future ensemble voting weights.
4. **False Alarm Recovery:** If the drift signal subsides without escalating to full drift, the background learner is seamlessly pruned, reclaiming heap memory.

Algorithmic Customization and Specialization

When a tree variant requires a reset or initialization, the intermediate layer handles the structural differences between algorithmic families cleanly:

- **Global Feature Subspaces (SRP):** The `RandomPatchesTree` variant relies on an atomic reference-counted pointer (`Arc<FeatureSubspace>`) to share its global feature mask. When a reset occurs, the middle layer samples a completely fresh subset of global indices and swaps the reference, minimizing allocation overhead across worker threads.
- **Local Feature Subspaces (ARF):** The `AdaptiveRandomTree` manages its feature subspace locally inside its own nodes during split evaluation. Consequently, it ignores external subspace arguments during a reset, initializing itself strictly using its internal structural configuration parameters.

Mondrian Tree Implementation

For Mondrian-based streaming ensembles (AMF), the system bypasses the `HTBasedLearner` manager and hooks directly into the `OnlineLearner` enum through the `MondrianTree` structure. Because the architecture is modular and uses a decoupled design, adding the Mondrian-based approach did not require altering a single line of Hoeffding-based logic or

dataflow code; it simply required implementing the standardized `OnlineLearnerTrait`, validating the true "plug-and-play" nature of the design.

3.2. Distributed Dataflow Pipeline

The core idea behind our design is to exploit *horizontal parallelism*: each tree in the ensemble is treated as an independent learner, and the workload is distributed by assigning different trees to different processing units (cores or hosts). This approach is inherently *embarrassingly parallel*, because the training of individual trees does not require synchronization among them. The only coordination point is the aggregation of predictions to produce the final ensemble output.

A naive implementation would centralize the aggregation step on a single node, creating a bottleneck. However, by fully embracing the dataflow model, we can decouple the distribution of instances from the aggregation of results, allowing both to be parallelized. The pipeline is structured as a sequence of transformations that naturally flow from one stage to the next, without requiring a global coordinator. This is achieved by carefully keying the data stream at different points:

1. First, each instance is replicated to all trees. The stream is keyed by `tree_id`, ensuring that all updates for a given tree are processed by the same worker.
2. After each tree produces its prediction, the stream is re-keyed by `instance_id`. This brings together the predictions from all trees for the same instance, enabling parallel aggregation across different instances.
3. Once all predictions for an instance are collected, the final prediction is computed and the result is forwarded to a centralized sink for accuracy tracking and output.

This design eliminates the need for a dedicated master node that collects and redistributes results; instead, the dataflow itself orchestrates the coordination. The result is a pipeline that scales with the number of available cores and can be distributed across multiple hosts with minimal overhead.

Overview The complete processing pipeline is composed of four logical stages plus a final stage of post-processing in Python, illustrated conceptually in Figure 3.2. In the following subsections, we describe each stage in detail, following a consistent format: we first explain the purpose, then the operations performed, and finally the mapping to Renoir operators.



Figure 3.2: Logical stages of the ensemble pipeline.

3.2.1. Stage 1: Input and Initialization

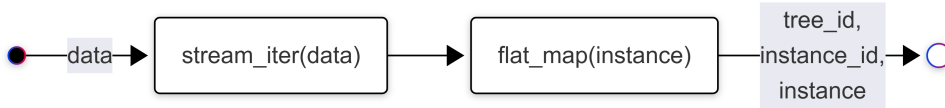


Figure 3.3: Renoir pipeline: Input and Initialization

Purpose. The first stage reads the input stream, parses instances, and initializes the global configuration. It also generates the initial replication of instances to all trees.

Operations. The input is a labeled dataset in ARFF format. The system reads each instance, converting it into a common representation that includes the feature vector, label, and a unique instance identifier. At the same time, the runtime configuration (number of trees, learner parameters, etc.) is loaded and made available to downstream operators.

Note that in this baseline we deliberately use a single entry node (the source runs on one host) to keep the design simple and to isolate the effects of the distributed learning pipeline. However, the architecture does not fundamentally rely on a single source: the same dataflow model would support parallel ingestion by partitioning the input stream across multiple sources (e.g., from multiple files or network feeds) without changing the downstream operators. Parallelizing the entry point would be a straightforward extension if the input rate becomes a bottleneck.

Renoir Implementation. The input is injected into the dataflow using a source operator. In our implementation, we use `stream_iter` to wrap an iterator over the dataset. For each instance, we associate an increasing `instance_id` and then `flat_map` it to `n_trees` copies, each paired with a distinct `tree_id`. This creates a stream of `(tree_id, instance_id, instance)` tuples.

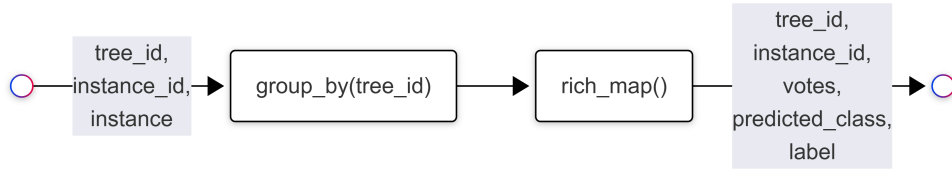


Figure 3.4: Renoir pipeline: Per-Tree Processing

3.2.2. Stage 2: Per-Tree Processing

Purpose. Each tree in the ensemble is trained and evaluated independently. This stage encapsulates the core learning logic for a single base learner (e.g., a Hoeffding Tree, Mondrian Tree, etc.). The tree maintains its internal state (statistics, split decisions) and processes each incoming instance in a test-then-train manner.

Operations. The stream is first partitioned by `tree_id` using `group_by`, ensuring that all tuples belonging to the same tree are routed to the same operator instance. For each tree, we initialize an `OnlineLearner` object that encapsulates the specific learning algorithm (determined by the ensemble type). For each incoming instance, the learner calls the `predict` method, and then the `train` method, outputting a tuple containing the `instance_id`, the votes vector containing the probabilities, the predicted class, and the actual label. The votes vector will then be used to compute the weighted voting if required, while the predicted class will then be used to compute the majority voting, and avoid its re-computation.

Renoir Implementation. This stage is implemented using `rich_map` on the partitioned stream, where the closure captures a per-tree state (`learner`)

Note that the output stream is still keyed by `tree_id` (since `group_by` preserves the key). The next stage will change the key to enable aggregation.

3.2.3. Stage 3: Per-Instance Aggregation

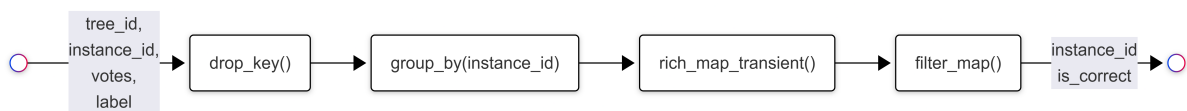


Figure 3.5: Renoir pipeline: Per-Instance Aggregation

Purpose. After each tree has produced its prediction for a given instance, we need to collect all these predictions and combine them into a single ensemble prediction. This

stage groups the stream by `instance_id` and aggregates the votes from all trees.

Operations. The stream is first re-keyed by `instance_id` using `group_by`. For each instance, we maintain a state that accumulates votes from the trees seen so far. The state includes:

- A counter `count` of how many trees have reported.
- A vector `combined_votes` that sums the class probabilities (optionally weighted by the tree’s cumulative accuracy).
- A sum of the tree depths for averaging.

When a new tuple arrives for the same `instance_id`, we update the state. Once the counter reaches the total number of trees (`n_trees`), we compute the final prediction (by taking the class with the highest accumulated probability), and output a result. Until that point, we emit `None` to indicate that aggregation is still ongoing.

Renoir Implementation. This stage is implemented with first `drop_key`, then `group_by`, followed by `rich_map_transient`. The `rich_map_transient` allows us to discard the state for a given key once the aggregation is complete.

`filter_map` then discards the `None` values, producing a stream of fully aggregated results.

Note that this implementation does not guarantee that instances are outputted in their original input order. This design choice was made intentionally to prioritize system throughput and scalability, avoiding idle stages.

3.2.4. Stage 4: Output and Data Sink

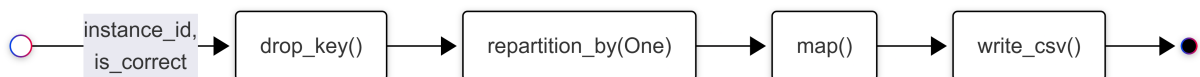


Figure 3.6: Renoir pipeline: Output

Purpose. The final streaming stage routes the fully aggregated results to a single worker node acting as the pipeline’s data sink. This provides a centralized point to finalize tracking, format records, and persist the execution output.

Operations. The stream of aggregated results is forced to a single partition using `repartition_by(Replication::One)`. On that single target node, we format the resulting data records to match the expected schema for storage.

While a production system might forward these predictions to a low-latency database, a message broker (e.g., Apache Kafka), or a network socket, this work implements a file-based sink. This choice serves as part of our evaluation framework, allowing downstream collection of metrics.

Renoir Implementation. Since the stream no longer requires key-based routing, we drop the active key. We then apply `repartition_by` to route all data items to a designated host (host 0), followed by a standard `map` operator to map the tuples into serializable output objects. Finally, the stream is persisted using Renoir’s native `write_csv` operator.

3.2.5. Evaluation Test Harness and Post-Processing

Rather than acting as a component of the live streaming dataflow architecture, the post-processing phase serves purely as an offline test harness designed to facilitate evaluation.

Once the dataflow pipeline terminates, the resulting CSV file containing the ensemble’s aggregated predictions is evaluated alongside metadata from the execution environment. This file is ingested by external Python scripts that compute global performance statistics (such as streaming accuracy, throughput, average depth of the trees) across independent operational runs. These scripts then automatically aggregate the metrics to generate the comparative summary tables and plots presented in Chapter 4.

3.3. Summary

The architecture achieves efficiency through careful layering of abstractions:

- **Unified learner interface:** Multiple ensemble methods coexist within a single codebase via a trait-based design, reducing implementation burden and enabling fair comparisons.
- **Dataflow-based coordination:** By keying the stream first by tree (for training) and then by instance (for aggregation), both stages parallelize without a central bottleneck.
- **Efficient state management:** Per-partition state is maintained in-place and discarded once complete, preventing memory leaks in long-running streams.

- **Flexible aggregation:** Voting strategies, drift detection mechanisms, and hyper-parameters are configurable orthogonal features.

This design enables the system to scale from a single core to distributed clusters while preserving the semantics of the ensemble methods.

4 | Experiments and Results

This chapter presents a comprehensive experimental evaluation of the proposed distributed ensemble methods implemented on the Renoir dataflow runtime. The evaluation is designed to validate the system’s viability across three critical dimensions.

First, we establish a baseline comparison against centralized implementations to quantify the throughput escalation and scalability afforded by the distributed architecture, while ensuring that predictive integrity is maintained. Second, we situate our work within the current research landscape by benchmarking it against a state-of-the-art distributed baseline, the Vertical Hoeffding Tree (VHT). This comparison not only assesses absolute performance in terms of accuracy and speed, but also evaluates the relative efficiency of the framework-level transition from sequential to distributed execution. Finally, we explore and study the architectural robustness of the system by analyzing how different hyperparameter configurations and execution settings influence model stability and performance under varying stream conditions.

Overall, this chapter seeks to address the following *Research Questions*:

- **Q1:** How does the proposed ensemble approach compare to the baseline in terms of predictive accuracy and computational efficiency?
- **Q2:** To what extent does the Renoir implementation improve computational throughput and scalability relative to a sequential execution?
- **Q3:** How does the proposed ensemble architecture perform in comparison to the state-of-the-art distributed VHT baseline in terms of predictive performance and raw throughput?
- **Q4:** How does the throughput speedup achieved by the proposed ensemble methods, when transitioning from a centralized to a distributed framework, compare to the speedup observed in existing distributed decision tree implementations?
- **Q5:** To what extent do hyperparameter configurations influence the model’s stability, and what are the optimal settings for diverse data stream conditions?

4.1. Experimental Setup

4.1.1. Hardware Setup

All experiments were conducted on a machine equipped with an AMD Ryzen 9 9950X processor (16 cores, 32 threads), 64 GB of RAM, running Fedora Server 42. This hardware configuration allows us to fully exploit parallelism in the proposed distributed implementation, while also providing a reliable baseline for throughput evaluation.

4.1.2. Software Setup

The Rust implementations were developed and executed using Rust version 1.95.0. For the MOA baseline, evaluation was conducted using version 24.07.0.

The River baseline was evaluated using version 0.24.2 on Python 3.14.4. This release incorporates our recent fixes that resolve inconsistencies identified in prior AMF implementations. Furthermore, to optimize computational throughput, depth updates within the Mondrian trees were explicitly disabled. Due to legacy dependency constraints, the Onelearn package was executed using Python 3.8.

4.1.3. Datasets Analysis

The experimental evaluation focuses on both real-world and synthetic datasets adopted in the SRP study [20]. These datasets are widely used in the data stream mining literature and are specifically designed to capture a variety of concept drift scenarios, including abrupt, gradual, and incremental changes.

The adoption of this benchmark suite ensures:

- **Consistency with prior work**, enabling direct comparison with existing results;
- **Coverage of diverse drift conditions**, which is essential for assessing the robustness of adaptive ensemble methods;
- **Reproducibility** of the experimental protocol.

In addition, a complementary evaluation was conducted using the datasets introduced in the VHT study [11].

Table 4.1 summarizes the main characteristics of all considered datasets. The *Drift* column indicates the type of concept drift, defined as follows:

Dataset	# Instances	# Features	Type	Drift	Classes
Airlines	539,383	7	Real	-	2
Elec	45,312	8	Real	-	2
Covtype	581,012	54	Real	-	7
Kdd99	4,898,431	41	Real	-	23
Internet ads	3,279	1,559	Real	-	2
Nomao	34,465	119	Real	-	2
Spam Corpus	9,324	39,917	Real	-	2
LED(a)	1M	24	Synthetic	A	10
LED(g)	1M	24	Synthetic	G	10
AGR(a)	1M	9	Synthetic	A	2
AGR(g)	1M	9	Synthetic	G	2
RBF(m)	1M	10	Synthetic	M	5
RBF(f)	1M	10	Synthetic	F	5
URT	1M	10, 50, 100, 500, 1k	Synthetic	<i>None</i>	2
SGZ	1M	500, 1k, 5k, 10k, 50k	Synthetic	<i>None</i>	2

Table 4.1: Summary of dataset characteristics

- **A**: Abrupt drift
- **G**: Gradual drift
- **M**: Moderate incremental drift
- **F**: Fast incremental drift

Real Datasets

The real-world datasets originate from diverse application domains and present varying levels of dimensionality and complexity.

Airlines This dataset is derived from a regression problem introduced by Ikonomovska and reformulated as a classification task. The objective is to predict whether a flight will be delayed based on scheduled departure information.

Electricity The Electricity dataset models the Australian New South Wales electricity market, where prices fluctuate based on supply and demand. The task is to predict whether the price will increase or decrease relative to a moving average over the previous 24 hours. The dataset exhibits strong temporal dependencies.

Forest Cover Type The Covtype dataset contains cartographic variables describing forest areas in the Roosevelt National Forest (Colorado). The task is to predict the forest

cover type among seven possible classes.

KDD99 Originally introduced for the KDD Cup 1999 competition, this dataset is used for network intrusion detection. The goal is to distinguish between normal and malicious network connections in a simulated military environment.

Internet ads This dataset addresses a binary classification task where the goal is to determine whether an image corresponds to an advertisement or not.

Nomao Nomao is a record linkage dataset, where each instance compares two entities (places). The objective is to determine whether they refer to the same real-world location.

Spam Corpus The SMS Spam Collection dataset consists of labeled text messages, where each message is classified as either legitimate (ham) or spam.

Synthetic Datasets

Synthetic datasets are generated algorithmically, allowing precise control over the data distribution and drift characteristics. This enables a more accurate assessment of how models react to controlled changes in the data stream.

LED(a) and LED(g) The LED dataset simulates a digit recognition task based on a 7-segment display. Although 24 binary features are provided, only 7 are relevant, while the remaining ones act as noise. Concept drift is introduced by dynamically changing which attributes are relevant. It is introduced every 250k instances.

AGR(a) and AGR(g) The Agrawal dataset models a loan approval task based on synthetic financial profiles. Drift is simulated by switching among multiple predefined classification functions, effectively altering the underlying decision logic over time. It is introduced every 250k instances.

RBF(m) and RBF(f) The Radial Basis Function dataset is a multi-class problem with non-linear decision boundaries defined by moving centroids in the feature space. Drift is introduced by gradually shifting these centroids, with different speeds controlling the drift intensity.

URT and SGZ These datasets, called *Dense* and *Sparse* respectively in the original VHT work [11], are characterized by the absence of concept drift and by non-redundant

feature spaces. Due to the lack of redundancy and noise, reducing the ensemble size typically leads much more to a degradation in predictive performance than to improvements in computational efficiency. However, although these datasets do not represent the primary target scenario of this thesis, they are briefly analyzed for a fair comparison.

- **Uniform Random Tree (URT)**: each instance consists of a fixed-length dense numeric vector, where features are independently sampled from a uniform distribution in $[0, 1]$. Class labels are generated using a random decision tree, constructed to ensure an approximately balanced class distribution. Datasets are generated with varying dimensionality (10 to 1000 features) to evaluate scalability in dense settings.
- **Sparse Gaussian Zipf (SGZ)**: each instance is represented as a high-dimensional sparse vector, analogous to a bag-of-words model, with only a small number of non-zero entries. The number of active features per instance is sampled from a clipped Gaussian distribution, while feature indices follow a Zipf-like distribution. Class labels are uniformly sampled, and feature distributions are conditioned on the class, ensuring discriminative patterns. Datasets range from 500 to 50,000 features, enabling the evaluation of performance in highly sparse scenarios.

4.1.4. Accuracy Parity Analysis and Validation

To rigorously assert that a newly proposed method achieves performance parity with an established baseline, conventional Null-Hypothesis Significance Testing (NHST) is inherently inadequate. In such frameworks, the null hypothesis posits that no difference exists between the two populations ($H_0 : \mu_1 = \mu_2$). Failing to reject this null hypothesis does not provide statistical evidence of equivalence; rather, it merely indicates an absence of sufficient evidence to detect a difference, often caused by low statistical power or small sample sizes [26].

To mathematically confirm that two predictive models yield equivalent performance metrics (e.g., accuracy), an equivalence testing paradigm must be adopted. The Two One-Sided Tests (TOST) procedure addresses this by inverting the traditional hypotheses, placing the burden of proof on demonstrating that the difference between two models falls within a strictly defined, clinically or empirically negligible range [25].

Mathematical Formulation Let μ_1 and μ_2 denote the true mean accuracies of the baseline model and the proposed distributed model, respectively. We define $\mu_d = \mu_1 - \mu_2$ as the true mean difference. An equivalence margin, $\Delta > 0$, is established as the maximum acceptable performance discrepancy that carries no practical significance in

the deployment environment.

The TOST framework decomposes the hypothesis of non-equivalence into two distinct composite null hypotheses (H_{01} and H_{02}), which are tested simultaneously:

$$H_{01} : \mu_d \leq -\Delta \quad \text{or} \quad H_{02} : \mu_d \geq \Delta \quad (4.1)$$

The alternative hypothesis (H_1) asserts that the performance difference is entirely contained within the bounded equivalence interval:

$$H_1 : -\Delta < \mu_d < \Delta \quad (4.2)$$

To reject the global null hypothesis of non-equivalence, both individual one-sided tests must be rejected at the significance level α . This is operationalized via two separate test statistics, t_1 and t_2 :

$$t_1 = \frac{\bar{X}_d - (-\Delta)}{SE} = \frac{\bar{X}_d + \Delta}{SE}, \quad t_2 = \frac{\bar{X}_d - \Delta}{SE} \quad (4.3)$$

where $\bar{X}_d$ represents the observed mean difference in accuracy across evaluation folds or bootstrap samples, and SE denotes the pooled standard error of the difference.

Decision Rule and Exact Distribution Assuming a Student's t -distribution (or a non-parametric equivalent if normality assumptions are violated), the respective p -values for the two one-sided tests are calculated as follows:

$$p_1 = P(T_{df} \geq t_1) \quad \text{and} \quad p_2 = P(T_{df} \leq t_2) \quad (4.4)$$

The overall p -value for the TOST procedure (p_{TOST}) is defined as the maximum of the two individual p -values:

$$p_{\text{TOST}} = \max(p_1, p_2) \quad (4.5)$$

If $p_{\text{TOST}} < \alpha$ (typically $\alpha = 0.05$), we reject the null hypothesis of non-equivalence at the $(1 - 2\alpha) \times 100\%$ confidence level, thereby concluding that the predictive integrity of the models is functionally identical within the margin Δ .

4.1.5. Reference Frameworks

MOA

Massive Online Analysis (MOA) is the de facto standard open-source framework for mining evolving data streams. It offers an extensive ecosystem of machine learning algorithms, including classification, regression, and clustering, alongside robust evaluation methodologies such as prequential and holdout evaluation [6].

Although MOA is primarily designed around a centralized execution model, it provides limited support for parallelism in specific algorithms. In particular, ARF includes an internal multi-threaded implementation that allows training its ensemble members concurrently across multiple cores. However, this capability is not uniformly available across the library: other algorithms, such as SRP, remain strictly single-threaded. In every case the degree of parallelism in MOA is algorithm-dependent and never extends to a general distributed execution model.

SAMOA

Scalable Advanced Massive Online Analysis (SAMOA) was originally conceived as the distributed counterpart to MOA, designed to provide a platform-independent abstraction layer for distributed stream mining across engines like Apache Flink or Apache Samza. Its goal was to bring the algorithmic richness of MOA to large-scale distributed environments.

However, the SAMOA project is currently inactive, with its development having stalled in 2019. More importantly for the scope of this research, it does not include implementations of the specific ensemble strategies considered in this work. This gap in the existing software landscape underscores the necessity for a new implementation that combines the rigorous algorithmic foundations of stream mining with modern, high-performance distributed computing paradigms.

River

River is a Python library designed for online machine learning. It aims to be the most user-friendly tool for performing machine learning on streaming data. River provides on-line implementations of numerous machine learning algorithms across various categories. Additionally, it supports dataset preprocessing and the creation of model pipelines. The library is continually evolving, regularly incorporating state-of-the-art algorithms.

The River library operates in a strictly single-threaded environment due to the inherent concurrency limitations of Python, specifically the Global Interpreter Lock (GIL).

Although River integrates extensions written in Cython and Rust to accelerate individual tree implementations, the overall execution speed remains fundamentally constrained by Python’s architecture.

Onelearn

ONE-shot LEARNing (Onelearn) is a lightweight Python package for online machine learning. It is notable for providing the first implementation of the Aggregated Mondrian Forest (AMF) algorithm, though the package has not been updated in several years.

To achieve high performance on large datasets, Onelearn leverages Numba, a Just-In-Time (JIT) compiler, combined with a Structure of Arrays data layout. This architecture delivers the computational speed of lower-level languages while preserving Python’s user-friendly interface. However, multithreading is not supported by the package.

4.2. Experimental Evaluation

4.2.1. Comparison with MOA

The following experimental analysis evaluates the performance of the implemented ensemble methods within the Renoir framework, comparing them against the state-of-the-art MOA library. This evaluation focuses specifically on Hoeffding Tree-based ensembles, namely the SRP and ARF algorithms. It should be noted that AMF is excluded from this comparison as it lacks an equivalent implementation in the current version of MOA.

Goal. The objective of this experiment is to perform a functional validation, ensuring that our distributed implementation maintains the predictive integrity of the original algorithms, and to quantify the throughput gains provided by our Rust-based distributed architecture.

Fairness of Comparison. To ensure a fair comparison, the algorithms within our architecture strictly adhere to the original logic and hyperparameters defined in the MOA reference source code, simply translated into Rust. These implementations were then integrated into the distributed dataflow model described in Section 3.2.

The comparison necessitates a distinction between the execution models of the two systems. While MOA is primarily sequential, ARF in MOA is capable of concurrent tree training via a shared-memory thread pool. Conversely, SRP in MOA is strictly single-threaded. In contrast, our system utilizes a distributed dataflow approach where

parallelism is inherent to the pipeline structure.

The comparison must account for the different execution models:

- **Single-Core Analysis (10 Trees):** We evaluate both MOA and Renoir using a single core¹. Since both systems are restricted to one core, the performance difference highlights the intrinsic efficiency of the Rust implementation and the Renoir runtime compared to the Java-based MOA.
- **Multi-Core Analysis (100 Trees):** This experiment focuses exclusively on ARF, as it is the only ensemble in MOA capable of exploiting multi-core parallelism. Including SRP in this context would be unproductive, as its throughput would scale inversely with the number of trees due to its single-threaded nature. By evaluating 100 trees across 32 cores, we can assess how our distributed dataflow model scales relative to a mature, centralized, and parallelized baseline.

Evaluation Strategy. The experimental parameters, selected to match standard literature benchmarks, are detailed in Table 4.2. To account for the non-deterministic nature of the algorithms, all Renoir experiments were repeated 10 times with unique seeds; results report the mean and standard deviation. MOA runs are reported as single values due to their deterministic nature in these settings.

Formal validation of predictive equivalence To formally verify functional replication across our entire work, a non-parametric Two One-Sided Tests (TOST) [26] procedure is utilized with an exact distribution method and a predefined equivalence margin $\Delta \pm 0.03$. This specific 3% threshold is standard in this environment and aligns with established empirical benchmarks in modern machine learning validation frameworks for establishing model equivalence [1, 25]. A rejection of the null hypothesis ($p_{\text{TOST}} < 0.05$) indicates that the accuracy profiles are practically equivalent within a 3% threshold.

Accuracy and Functional Validation

The accuracy results for ensembles of 10 and 100 base models are summarized in Table 4.3 and Table 4.4, respectively.

The results demonstrate that our implementation achieves predictive parity with MOA. Minor deviations are expected and attributable to several factors:

¹Renoir actually uses multiple threads to handle the pipelining, but the execution of the test then train logic is ensured to be sequential

Drift detection enabled	drift_detection	<i>true</i>
Number of trees in the ensemble	n_trees	<i>10, 100</i>
Max number of bins in histograms	max_bins	<i>128</i>
Grace period	n_min	<i>50</i>
Hoeffding bound's delta	delta	<i>0.01</i>
ADWIN delta warning	adwin_delta_warning	<i>1e-4</i>
ADWIN delta drift	adwin_delta_drift	<i>1e-5</i>
Hoeffding bound's Tau	tau	<i>0.05</i>
Subspace size of each learner	features_patch	<i>60%</i>
Poisson average rate	lambda	<i>6.0</i>
Voting mechanism	voting	<i>Weighted</i>
Numeric estimator	numeric_estimator	<i>Gaussian</i>

Table 4.2: ARF and SRP parameters

Dataset	MOA ARF	Renoir ARF	MOA SRP	Renoir SRP
Airlines	65.79	66.8 $\pm$ 0.02	66.77	66.70 $\pm$ 0.02
Elec	88.72	88.43 $\pm$ 0.07	88.82	88.28 $\pm$ 0.16
Covtype	94.69	94.52 $\pm$ 0.03	95.25	94.57 $\pm$ 0.03
Kdd99	99.98	99.99 $\pm$ 0.00	99.99	99.99 $\pm$ 0.00
Internet ads	99.73	99.79 $\pm$ 0.04	99.76	99.81 $\pm$ 0.01
Nomao	97.06	96.70 $\pm$ 0.09	97.23	96.61 $\pm$ 0.03
Spam Corpus	96.21	96.47 $\pm$ 0.15	95.97	96.38 $\pm$ 0.16
LED(a)	73.94	73.97 $\pm$ 0.01	73.58	72.17 $\pm$ 0.36
LED(g)	73.01	73.10 $\pm$ 0.01	72.41	70.34 $\pm$ 0.71
AGR(a)	89.92	94.66 $\pm$ 0.03	90.80	90.53 $\pm$ 1.51
AGR(g)	88.15	91.88 $\pm$ 0.03	87.76	88.41 $\pm$ 1.25
RBF(m)	84.49	84.47 $\pm$ 0.03	83.28	82.33 $\pm$ 0.01
RBF(f)	70.71	70.76 $\pm$ 0.05	70.82	67.08 $\pm$ 0.10

Table 4.3: Hoeffding Tree-Based Ensembles accuracy (%) using 10 base models

- **Floating-Point Precision:** Small differences in how Rust and Java handle IEEE 754 arithmetic, particularly under different optimization levels.
- **PRNG Differences:** Even with the same seed, the underlying algorithms for PseudoRandom Number Generator differ between the languages.
- **Instance Ordering:** In a distributed dataflow, the order in which instances reach the ensemble aggregator may vary due to network or scheduling jitter, affecting the final model state in non-commutative operations.
- **Implementation details:** Some minor implementation details may exist.

The TOST framework successfully rejected the null hypothesis of non-equivalence for both algorithms, mathematically confirming that the transition to a distributed archi-

Dataset	MOA ARF	Renoir ARF	MOA SRP	Renoir SRP
Airlines	66.41	67.36 $\pm$ 0.00	68.56	67.49 $\pm$ 0.01
Elec	89.67	89.27 $\pm$ 0.02	89.85	89.54 $\pm$ 0.03
Covtype	94.96	94.99 $\pm$ 0.00	95.34	95.09 $\pm$ 0.01
Kdd99	99.98	99.99 $\pm$ 0.00	99.98	99.99 $\pm$ 0.00
Internet ads	99.69	99.82 $\pm$ 0.00	99.72	99.82 $\pm$ 0.00
Nomao	97.19	96.9 $\pm$ 0.01	97.38	96.89 $\pm$ 0.01
Spam Corpus	97.31	97.38 $\pm$ 0.02	97.43	97.19 $\pm$ 0.01
LED(a)	73.96	74.01 $\pm$ 0.01	74.02	73.95 $\pm$ 0.00
LED(g)	73.09	73.18 $\pm$ 0.00	73.23	73.07 $\pm$ 0.01
AGR(a)	87.92	94.88 $\pm$ 0.01	92.86	93.45 $\pm$ 0.06
AGR(g)	82.28	92.26 $\pm$ 0.00	89.65	90.72 $\pm$ 0.13
RBF(m)	86.95	87.35 $\pm$ 0.01	86.03	86.42 $\pm$ 0.00
RBF(f)	76.29	76.9 $\pm$ 0.01	76.37	75.03 $\pm$ 0.01

Table 4.4: Hoeffding Tree-Based Ensembles accuracy (%) using 100 base models

texture does not compromise the predictive integrity of the models. These findings are further supported by the temporal behavior of the models;

Figure 4.1 provides two illustrative examples of how accuracy evolves over time under different streaming conditions:

1. The first example, involving the real-world **Airlines** dataset (Figure 4.1a), reveals high volatility in the first half of the stream. Multiple concept drifts cause frequent spikes in the prequential accuracy, which naturally constrains the final cumulative accuracy to a lower threshold.
2. The second example utilizes a synthetic **LED(a)** dataset (Figure 4.1b) with controlled abrupt drifts every 250k instances. Here, we observe an early drift likely associated with initial model assessment and warm-up. Subsequently, the prequential accuracy captures the injected drifts exactly where expected; the metrics drop immediately following a drift event and recover quickly thereafter, demonstrating the system’s ability to rapidly re-establish predictive performance.

Throughput Analysis

The core contribution of our implementation lies in its significant performance gains over the state-of-the-art MOA framework. Table 4.5 presents the throughput using 10 learners on a single core to establish a baseline of raw execution efficiency, while Table 4.6 scales the experiment to a 100-learner ensemble across 32 cores to evaluate parallel scalability.

Dataset	MOA ARF	Renoir ARF	Speedup	MOA SRP	Renoir SRP	Speedup
Airlines	4,523	161,594 $\pm$ 1,090	35.73	5,305	159,196 $\pm$ 2,190	30.01
Elec	15,735	155,598 $\pm$ 3,454	9.89	10,357	137,620 $\pm$ 3,529	13.29
Covtype	9,146	71,636 $\pm$ 1,277	7.83	5,435	70,591 $\pm$ 1,535	12.99
Kdd99	28,930	88,823 $\pm$ 822	3.07	22,402	83,247 $\pm$ 2,560	3.72
Internet ads	1,380	13,577 $\pm$ 216	9.84	558	13,607 $\pm$ 130	24.39
Nomao	5,418	42,131 $\pm$ 569	7.78	3,327	42,860 $\pm$ 955	12.88
Spam Corpus	7	184 $\pm$ 6	29.29	6	186 $\pm$ 9	31.00
LED(a)	22,141	43,530 $\pm$ 1,565	1.97	12,396	51,674 $\pm$ 954	4.17
LED(g)	21,235	42,722 $\pm$ 1,717	2.01	12,275	49,770 $\pm$ 573	4.05
AGR(a)	17,067	188,432 $\pm$ 1,652	11.04	10,318	148,032 $\pm$ 2,789	14.35
AGR(g)	14,877	163,155 $\pm$ 1,012	10.97	8,867	141,077 $\pm$ 1,352	15.91
RBF(m)	20,005	88,036 $\pm$ 633	4.40	11,043	86,647 $\pm$ 560	7.85
RBF(f)	19,091	83,004 $\pm$ 329	4.35	9,855	81,414 $\pm$ 462	8.26

Table 4.5: Hoeffding Tree-Based Ensembles throughput (instances/second) using 10 base models and 1 core

Dataset	MOA ARF	Renoir ARF	Speedup
Airlines	8,177	171,367 $\pm$ 2,729	20.96
Elec	11,808	159,786 $\pm$ 4,926	13.53
Covtype	5,212	64,560 $\pm$ 2,128	12.39
Kdd99	5,555	79,746 $\pm$ 3,731	14.36
Internet ads	1,216	12,942 $\pm$ 491	10.65
Nomao	4,973	45,880 $\pm$ 1,127	9.23
Spam Corpus	<i>Crash</i>	857 $\pm$ 8	∞
LED(a)	6,058	36,385 $\pm$ 1,000	6.01
LED(g)	6,202	38,080 $\pm$ 576	6.14
AGR(a)	8,654	188,841 $\pm$ 2,282	21.82
AGR(g)	8,269	169,939 $\pm$ 1,227	20.55
RBF(m)	7,271	81,135 $\pm$ 743	11.16
RBF(f)	8,083	79,911 $\pm$ 574	9.89

Table 4.6: Hoeffding Tree-Based Ensembles throughput (instances/second) using 100 base models and 32 cores

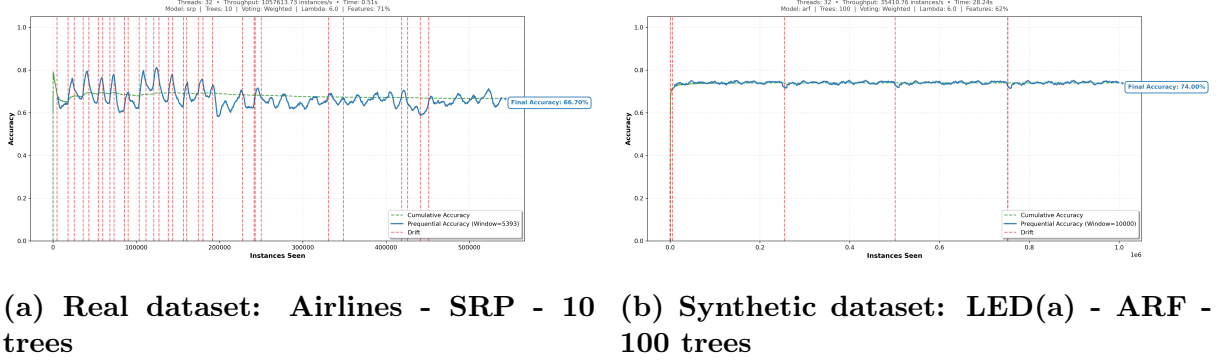


Figure 4.1: Accuracy over time: notable results.

Analysis of Results. The experimental results reveal that Renoir consistently outperforms MOA, with speedup factors influenced by feature dimensionality, the total volume of processed instances, and architectural scaling overhead.

In the single-core configuration (Table 4.5), which establishes a baseline of 10 trees per core, Renoir achieves substantial speedups, peaking at $35.73\times$ on the **Airlines** dataset. Since parallelism is disabled, these gains are primarily attributable to Rust’s zero-cost abstractions and efficient memory management. We observe a strong correlation between **feature dimensionality** and speedup: in high-dimensional datasets like **Internet ads** (1,559 features) and **Spam Corpus** (39,917 features), Renoir exhibits its highest relative gains ($24.39\times$ and $31.00\times$ for SRP, respectively). This suggests that MOA’s reliance on heap-allocated objects for feature vectors becomes a critical bottleneck as dimensionality grows.

The **total number of instances** also plays a role in the relative performance gap. For the **Kdd99** dataset, which contains nearly 4.9M instances, the speedup is notably lower ($3.07\times$ and $3.72\times$). This behavior is consistent with the JVM’s Just-In-Time compilation strategy; on very long-running streams, the JVM has sufficient time to identify and optimize "hot" code paths, allowing MOA to narrow the performance gap compared to the native, pre-compiled execution of Renoir.

While the specific type of **concept drift** (abrupt, gradual, or incremental) does not appear to significantly alter the speedup ratio, its presence necessitates frequent model updates which further highlight Renoir’s efficiency in handling the heavy control-flow logic of tree restructuring compared to the higher-latency branching found in the JVM implementation.

The transition to the 32-core configuration (Table 4.6) provides a deeper insight into multi-threaded orchestration. In this setup, the ensemble size is increased to 100 trees

across 32 threads, which actually reduces the **computational load per thread** from 10 trees (in the single-thread baseline) to 3.125 trees. Despite this "lighter" per-core workload, MOA's relative performance degrades, causing the speedup for Renoir to increase significantly (e.g., rising from $11.04\times$ to $21.82\times$ on **AGR(a)**).

This trend demonstrates that MOA encounters a *concurrency ceiling*. Even when provided with more relative hardware resources per learner, the JVM-based implementation suffers from high orchestration overhead, likely due to lock contention in its thread pool and the increased complexity of Garbage Collection across 32 active threads. Renoir's dataflow model, conversely, scales much more gracefully, effectively translating the increased hardware availability into higher throughput.

Finally, the robustness of the implementation is highlighted by the **Spam Corpus** dataset. Despite reducing the feature patch to 10%, the MOA implementation consistently crashed or timed out under the combined memory pressure of the 100-tree ensemble and multi-threaded execution. In contrast, Renoir successfully processed the stream, demonstrating superior stability and memory safety in high-dimensional, parallel scenarios.

4.2.2. Comparison with River/Onelearn

The following experimental analysis evaluates both the predictive accuracy and the computational throughput of the implemented ensemble method AMF within the Renoir framework, comparing it against the state-of-the-art Onelearn implementation. Evaluations of ARF and SRP were not conducted in this environment because the MOA framework provides significantly faster baseline implementations than River. Furthermore, as demonstrated by the AMF throughput evaluation in Table 4.7, River is much slower than Onelearn, establishing the latter as the preferred, more challenging baseline for the subsequent accuracy and performance evaluations.

Dataset	River AMF	Onelearn AMF	Speed-up
LED(a)	424.33 ± 2.13	2799.32 ± 1.15	6.60
LED(g)	401.24 ± 34.32	2795.14 ± 4.34	6.97
AGR(a)	382.75 ± 28.14	3220.26 ± 10.18	8.41
AGR(g)	393.10 ± 0.73	3231.57 ± 6.79	8.22
RBF(m)	544.99 ± 6.17	2611.34 ± 6.21	4.79
RBF(f)	498.31 ± 3.16	2395.48 ± 7.36	4.81

Table 4.7: River vs Onelearn throughput with 10 trees

Goal. This experiment aims to confirm that our Rust-based system preserves the predictive accuracy of the original algorithm, while also highlighting the throughput advantages gained from a multithreaded approach.

Fairness of Comparison. To ensure a fair evaluation between our Renoir dataflow architecture and Onelearn’s purely algorithmic approach, we loaded the entire dataset into memory for both implementations, thereby eliminating I/O bottlenecks. Because the Mondrian process is highly sensitive to feature magnitudes, we applied Min-Max scaling to the dataset using a batched approach. While streaming normalization is feasible, decoupling this preprocessing step ensures that scaling overhead does not introduce artifacts into our throughput measurements. Additionally, one-hot encoding was uniformly applied to address the limitations of Mondrian trees with categorical data.

Finally, all comparisons were conducted using an ensemble of 10 trees restricted to a single CPU core. This configuration eliminates the influence of multi-threading and specifically highlights the raw computational efficiency differences between Numba (utilized by Onelearn) and our native Rust implementation.

Evaluation Strategy. The specific configuration parameters for this experiment are identical to those presented in Table 4.8. Finally, to ensure the statistical reliability of the measurements, each experiment was executed ten independent times, with all reported performance metrics representing the mean and standard deviation across these iterations.

Formal validation of predictive equivalence To formally verify functional replication across our entire work, exactly like for the MOA comparison at Section 4.2.1, a non-parametric Two One-Sided Tests (TOST) procedure is utilized.

Step parameter	step	<i>1.0</i>
Dirichlet parameter	dirichlet	<i>0.5</i>
Random generator seed	seed	<i>0, 1, 2, 3, 4, 5, 6, 7, 8, 9</i>
Split pure nodes	split_pure	<i>false</i>
Number of trees	n_trees	<i>10</i>

Table 4.8: AMF parameters

Accuracy and Functional Validation

The accuracy results for the 10-tree ensemble are reported in Table 4.9. As shown in the table, the difference in mean accuracy across most datasets remains consistently below 1%, except for the LED(a), LED(g), and covTypeNorm datasets. This tight margin

of error confirms that our Rust implementation correctly and reliably replicates the core AMF algorithm.

Formally, the TOST framework successfully rejected the null hypothesis of non-equivalence for the algorithm, mathematically confirming the predictive equivalence of the models.

The minor discrepancies in accuracy between the implementations can be attributed to two primary factors:

- **PseudoRandom Number Generator (PRNG) Differences:** Because the splitting mechanism in the Mondrian process is an inherently stochastic event, variations in the underlying random number generators across different languages heavily influence the tree structure and, consequently, the final accuracy.
- **Floating-Point Precision:** While Onelearn utilizes 32-bit floating-point numbers (`f32`), River employs 64-bit precision (`f64`). We opted to natively use `f64` in our Rust implementation to ensure maximum numerical stability and precision.

Dataset	Onelearn AMF	Renoir AMF
Airlines	62.24 ± 0.24	62.53 ± 0.12
Elec	84.15 ± 0.47	84.15 ± 0.15
Covtype	88.86 ± 0.17	92.52 ± 0.06
Kdd99	99.72 ± 0.00	99.72 ± 0.00
Internet ads	82.66 ± 0.09	82.72 ± 0.07
Nomao	87.30 ± 0.08	87.43 ± 0.05
Spam Corpus	93.91 ± 0.35	93.73 ± 0.23
LED(a)	59.24 ± 0.88	61.89 ± 0.11
LED(g)	59.24 ± 0.88	61.90 ± 0.11
AGR(a)	61.95 ± 0.39	62.17 ± 0.06
AGR(g)	61.96 ± 0.39	62.17 ± 0.06
RBF(m)	64.55 ± 0.18	65.30 ± 0.05
RBF(f)	44.09 ± 0.29	44.75 ± 0.03

Table 4.9: AMF accuracy (%) using 10 base models

Dataset	Onelearn AMF	Renoir AMF	Speedup
Airlines	540 ± 3	879 ± 2	1.63
Elec	$4,259 \pm 28$	$77,674 \pm 1,903$	18.24
Covtype	$2,687 \pm 16$	$13,111 \pm 23$	4.88
Kdd99	$1,742 \pm 22$	$15,056 \pm 66$	8.64
Internet ads	502 ± 2	521 ± 2	1.04
Nomao	$2,618 \pm 4$	$8,155 \pm 71$	3.12
Spam Corpus	7 ± 0	6 ± 0	0.87
LED(a)	$2,799 \pm 1$	$20,137 \pm 19$	7.19
LED(g)	$2,795 \pm 4$	$20,127 \pm 20$	7.20
AGR(a)	$3,220 \pm 10$	$18,666 \pm 347$	5.80
AGR(g)	$3,232 \pm 7$	$18,752 \pm 126$	5.80
RBF(m)	$2,611 \pm 6$	$14,003 \pm 42$	5.36
RBF(f)	$2,395 \pm 7$	$12,756 \pm 77$	5.33

Table 4.10: AMF throughput (instances/second): 10 base models and 1 core

Analysis of Results

The throughput evaluation demonstrates that the Rust implementation outperforms the baseline on the majority of the datasets, with the notable exception of the **Spam Corpus** (which yielded a speedup of $0.87\times$). This specific case highlights an expected architectural trade-off triggered by the extreme dimensionality of the dataset (39,917 features). Our Rust implementation utilizes an Array of Structures (AoS) memory layout, which suffers from poor cache locality in single-threaded execution when handling massive feature vectors. In contrast, the Onelearn baseline employs a Struct of Arrays (SoA) layout. The SoA approach is explicitly optimized for single-threaded execution because it maximizes CPU cache efficiency by loading only the contiguous array of the specific feature being evaluated. In highly dimensional contexts like the **Spam Corpus**, this cache-friendly memory access pattern grants the baseline a significant hardware-level advantage.

Despite this expected single-core overhead, a distinct pattern emerges from the broader results: the relative speedup achieved by the native Rust implementation is highly sensitive to the dimensionality of the dataset. Specifically, lower-dimensional feature spaces yield significantly higher throughput speedups.

The primary computational bottleneck in the AMF algorithm is the recursive calculation of the bounding box distances between an incoming sample and the current node, traversing down to the leaf. As the number of features increases, these array-based com-

putations become inherently more expensive. In this context, Rust’s architecture offers a distinct advantage. By utilizing idiomatic iterators, Rust performs **bounds check elision**, eliminating the overhead of runtime index boundary verifications, which enables highly optimized, zero-cost abstractions for array and slice computations.

Furthermore, the inherent nature of the Mondrian process leads to frequent node splitting. Consequently, processing a high volume of instances inevitably increases the overall depth of the trees. While this increased depth incrementally degrades throughput, the dimensionality of the feature space remains the dominant factor. The AMF algorithm is particularly susceptible to the curse of dimensionality [23]; as the number of features grows, the sample space becomes increasingly sparse, resulting in vastly larger tree structures that inherently degrade traversal performance.

Finally, because the algorithm currently lacks a mechanism for tree pruning or node eviction to adapt to concept drift, obsolete historical data persists indefinitely within the Mondrian tree structures. This unchecked structural growth not only hinders the accuracy adaptability discussed previously, but also progressively exacerbates the throughput bottleneck, as the trees continuously increase in depth over the data stream’s lifetime.

4.2.3. Remote execution

Goal. The primary objective of this experiment is to validate the performance of the proposed solution in a distributed environment. By transitioning from a local shared-memory context to a networked execution model, we aim to quantify the impact of communication overhead on overall throughput.

Methodology. For this evaluation, we selected the most computationally demanding scenario: an ARF ensemble with 100 base models and an AMF ensemble restricted to 20 base models, both utilizing 32 logical cores. ARF was chosen as the representative algorithm due to its structural similarity to SRP in terms of parallelization requirements. The restriction on the AMF ensemble was necessary due to hardware memory constraints. To maintain hardware consistency while introducing network latency, we utilized the same physical machine described in the previous sections, logically partitioned into two distinct worker nodes, where each node was allocated 16 logical cores. Communication between these nodes was established via SSH through the Renoir framework, utilizing the loopback interface as a network boundary. This setup allows us to simulate a multi-node deployment, forcing the system to serialize data and manage distributed coordination.

Analysis of Results. Table 4.11 presents the comparison between the *Parallel Throughput* (local shared-memory execution) and the *Distributed Throughput* (distributed SSH-based execution) for ARF, while Table 4.12 details this comparison for AMF.

Dataset	Parallel Throughput	Distributed Throughput	Efficiency (%)
Airlines	171,413.31	151,850.83	88.59
Elec	168,822.75	107,791.60	63.85
Covtype	65,363.66	53,478.58	81.82
Kdd99	84,821.63	67,593.69	79.69
Internet ads	12,737.51	4,462.27	35.03
Nomao	46,744.94	34,136.17	73.03
Spam Corpus	857.77	231.50	26.99
LED(a)	36,123.37	34,988.43	96.86
LED(g)	37,060.33	35,995.73	97.13
AGR(a)	189,350.97	138,006.76	72.88
AGR(g)	169,464.21	131,969.86	77.87
RBF(m)	81,240.95	75,245.89	92.62
RBF(f)	79,860.36	74,369.78	93.12

Table 4.11: ARF throughput efficiency in remote execution using 100 base models and 32 cores

In the case of ARF, the results reveal a strong inverse correlation between the dimensionality of the dataset and the resulting execution efficiency. As seen in Table 4.1, datasets with a limited feature space, such as **Airlines** (7 features) or the synthetic LED and RBF suites (10–24 features), maintain high efficiency levels, often exceeding 90%. In these scenarios, the communication overhead remains negligible compared to the computational work of the ensemble.

Conversely, efficiency decreases as dimensionality increases, reaching its lowest points with **Internet ads** (1,559 features) and **Spam Corpus** (39,917 features). This indicates that the cost of serializing large feature vectors and transmitting them via SSH becomes the primary bottleneck in high-dimensional contexts. However, it is important to note that when comparing these figures to the benchmarks obtained in Section 4.2.1, the throughput remains significantly higher than the MOA baseline. This demonstrates that our solution provides a substantial performance improvement over state-of-the-art tools even in a real-world distributed scenario where network overhead is present.

Dataset	Parallel Throughput	Distributed Throughput	Efficiency (%)
Airlines	2,412.38	2,312.27	95.85
Elec	206,806.96	149,256.15	72.17
Covtype	38,869.59	36,677.92	94.36
Kdd99	42,660.08	41,935.18	98.30
Internet ads	1,126.14	1,021.58	90.72
Nomao	27,265.26	24,507.98	89.89
Spam Corpus	8.75	9.05	103.43
LED(a)	50,461.01	47,034.65	93.21
LED(g)	50,927.58	47,716.88	93.70
AGR(a)	49,800.59	46,382.16	93.14
AGR(g)	49,711.61	46,820.43	94.18
RBF(m)	38,560.65	36,542.91	94.77
RBF(f)	36,807.35	34,753.73	94.42

Table 4.12: AMF throughput efficiency in remote execution using 20 base models and 32 cores

Unlike ARF, the AMF ensemble behaves differently under distributed execution because its 20 base models are mapped onto 32 logical cores. With more cores than trees, extra threads handle data serialization and network communication concurrently. This effectively masks communication latency during the computationally intensive distance calculations, limiting the overall performance drop to approximately 10% (maintaining roughly 90% efficiency).

However, extreme dataset configurations alter this baseline. The **Elec** dataset is a notable exception, dropping to 72.17% efficiency. Its highly restricted feature space (14 features after one-hot encoding) causes distance calculations to finish almost instantaneously, leaving computational threads idle while waiting for data transmission. This bottleneck is further exacerbated by the low number of samples, which prevents the trees from growing large enough to increase the computational load. Conversely, the high-dimensional **Spam Corpus** dataset sees virtually no performance degradation. Its vast feature space requires substantial computation time per tree, allowing background threads to fully absorb network overhead without stalling the ensemble.

4.2.4. Scalability Analysis

In this subsection, we analyze the scalability of the proposed system, evaluating how its performance evolves as we increase the number of processing cores under diverse workloads.

Goal. Our primary objective is to quantify the efficiency gains achieved through parallel execution and to identify the hardware and algorithmic bottlenecks that influence throughput as resources scale.

Methodology. To evaluate horizontal scalability, we conducted experiments varying the number of allocated CPU cores $n \in \{1, 2, 4, 8, 16, 32\}$ across all algorithms. For ARF and SRP, we maintain a fixed ensemble size of 100 trees, isolating parallelization efficiency from the computational cost of the model itself. AMF, however, imposes a stricter constraint: its bounding-box partition structure incurs a substantially larger per-tree memory footprint, causing out-of-memory failures at 100 trees on the available hardware. Consequently, AMF experiments use a fixed ensemble of 20 trees. This difference in ensemble size means that raw throughput figures are **not directly comparable** across algorithms; we therefore report parallel efficiency η for ARF and SRP, and speedup for AMF, as each metric isolates parallelization behavior independently of ensemble size.

Speedup is defined as:

$$\text{Speedup}(n) = \frac{\text{Throughput}(n)}{\text{Throughput}(1)} \quad (4.6)$$

where $\text{Throughput}(n)$ is the number of instances processed per second using n cores. Parallel efficiency is defined as:

$$\eta(n) = \frac{\text{Speedup}(n)}{n} = \frac{\text{Throughput}(n)}{n \times \text{Throughput}(1)} \quad (4.7)$$

Our hardware configuration (16 physical cores and 32 logical threads via Simultaneous MultiThreading (SMT)) allows us to assess the transition from physical core saturation to logical thread exploitation across all algorithms.

Decentralized Aggregation Strategy. A significant challenge in parallelizing ensemble methods is the final aggregation (voting) stage. Traditional implementations often suffer from a "many-to-one" bottleneck where a single node must serialize and process results from all workers. As detailed in Section 3.2, we mitigate this by leveraging a

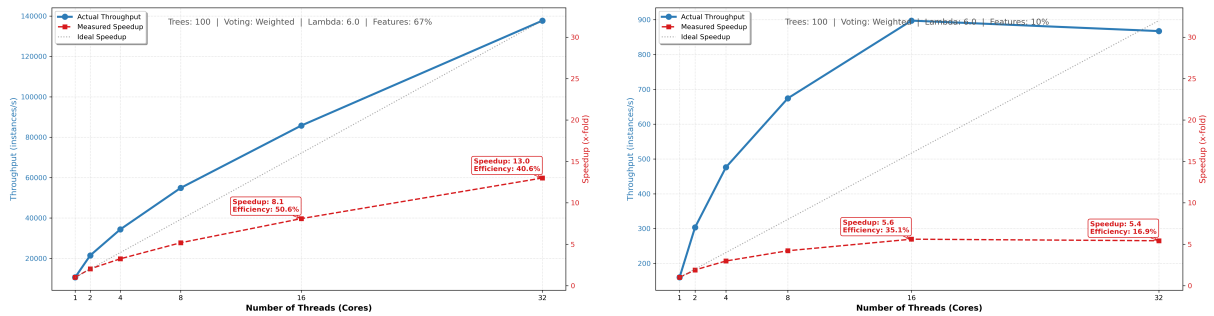
`group_by(instance_id)` operation. This strategy decentralizes the aggregation: votes for different instances are partitioned across the available workers, ensuring that communication and computation loads remain balanced across the dataflow graph as n increases.

ARF and SRP: Analysis of Efficiency and Dataset Characteristics.

The experimental results in Table 4.13 reveal that Renoir achieves consistent scaling across different ensemble methods and datasets, with Figure 4.2 illustrating two representative examples of scaling behavior:

- **Algorithm Consistency (ARF vs. SRP):** We note that the efficiency metrics for ARF and SRP are remarkably similar across almost all datasets. For example, on `Covtype` at 16 cores, ARF achieves 48.9% while SRP reaches 50.01%. This suggests that the bottleneck in scaling is not the specific logic of the ensemble method, but rather the underlying dataflow orchestration and memory access patterns managed by the Renoir runtime.
- **Physical vs. Logical Core Utilization:** Efficiency stays near the 50% threshold when utilizing the 16 physical cores, however, as the system scales to 32 logical threads, efficiency drops by approximately 10%. This indicates that while Renoir successfully extracts additional performance from SMT, shared execution resources (ALUs, L1/L2 caches) between logical pairs on a single physical core prevent ideal scaling.
- **Memory-Bound Bottlenecks and Regressions:** The impact of feature dimensionality remains the most significant factor in parallel efficiency. Low-dimensional datasets like `AGR(g)` (Figure 4.2a) maintain higher efficiency because smaller feature vectors allow for better L3 cache residency. In contrast, the `Spam Corpus` (Figure 4.2b) exhibits a unique behavior: its throughput at 32 logical threads is slightly lower than at 16 physical cores. With 39,917 features per instance, the massive memory footprint likely causes severe memory bus contention and cache thrashing; in this extreme case, the overhead of managing additional logical threads exceeds the marginal computational gain, leading to a negative scaling return.
- **Effect of Stream Volume:** Datasets with high instance counts, such as `Kdd99` and the synthetic streams, demonstrate more stable efficiency. This suggests that Renoir's decentralized aggregation (via `group_by`) is particularly effective at maintaining throughput "steady-states" once the dataflow pipeline is fully saturated.

Overall, these results confirm that Renoir effectively transforms hardware scaling into processing throughput for most scenarios, while also highlighting the physical memory



(a) Compute-bound scaling: AGR(g) - SRP (b) Memory-bound regression: Spam Corpus - ARF

Figure 4.2: Scaling behaviors: notable results.

limits encountered when processing high-dimensional data in parallel.

Dataset	ARF 16 core	ARF 32 core	SRP 16 core	SRP 32 core
Airlines	47.7	37.7	48.8	38.6
Elec	46.5	37.0	45.8	34.4
Covtype	48.9	36.9	50.0	36.0
Kdd99	48.6	26.9	51.7	36.7
Internet ads	45.1	28.6	47.3	30.2
Nomao	46.7	34.9	50.8	36.4
Spam Corpus	35.1	16.9	37.2	17.4
LED(a)	50.1	37.8	48.7	35.6
LED(g)	48.9	37.0	48.0	35.7
AGR(a)	48.7	38.0	49.8	39.1
AGR(g)	49.4	37.6	50.6	40.6
RBF(m)	51.8	40.3	51.1	39.7
RBF(f)	49.5	38.8	49.6	38.2

Table 4.13: Efficiency (%) for ARF and SRP, 100-tree ensemble.

AMF: Analysis of Efficiency and Dataset Characteristics

The experimental results, reported in Table 4.14, demonstrate a consistent speedup across most datasets; however, performance does not scale linearly with the addition of threads. While the speedup remains relatively close to the ideal when scaling from two to four threads, efficiency noticeably degrades and deviates further from the linear trajectory as the thread count increases to eight and sixteen.

A primary factor influencing this bottleneck is data dimensionality. For high-dimensional datasets such as Internet ads and Spam Corpus, the performance gains introduced by

additional threads are significantly diminished. These datasets possess a high feature-to-sample ratio: 39,917 features to 9,324 samples for the **Spam Corpus**, and 1,559 features (prior to one-hot encoding) to 3,279 samples for **Internet ads**. Processing these wide feature spaces across multiple threads likely saturates memory bandwidth and increases cache contention, inherently limiting parallel efficiency. As demonstrated in Figure 4.3b, the speedup for the **Spam Corpus** severely plateaus as early as four to eight threads. Conversely, execution on the **RBF(m)** dataset continues to scale, achieving a peak speedup of 17.66 at 32 threads (Figure 4.3a).

Furthermore, the system fails to achieve an ideal linear speedup due to the following algorithmic and hardware bottlenecks:

- **Amdahl’s Law [2]:** The maximum theoretical speedup is bounded by the sequential fraction of the pipeline. System-wide synchronization points, specifically data broadcasting and final prediction aggregation, dominate execution time as the thread count increases, establishing a hard performance cap.
- **Simultaneous Multithreading (SMT) Limitations:** The underlying hardware utilizes 16 physical cores exposing 32 logical threads. When scaling beyond 16 threads, paired logical threads compete for identical physical execution units (e.g., ALUs and cache). Consequently, 32 logical threads cannot deliver the computational throughput of 32 dedicated physical cores.
- **Ensemble Size Constraint:** The ensemble is strictly limited to 20 trees, restricting the distribution of the primary computational workload to a maximum of 20 cores. The remaining 12 cores execute only lightweight auxiliary dataflow tasks, such as data routing and serialization. This severe workload asymmetry leaves a significant portion of the hardware capacity underutilized.

Nevertheless, a positive scalability trend is generally maintained provided the number of active cores does not exceed the number of trees in the ensemble.

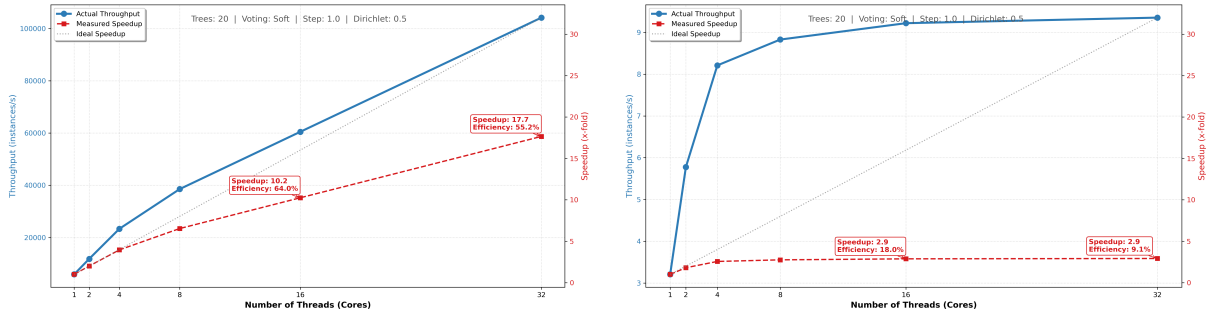
Scalability Comparison with MOA

In this section, we evaluate the scalability and robustness of **Renoir** by comparing it against the centralized parallel implementation provided by **MOA**. The objective is to determine how our distributed-oriented architecture performs relative to a traditional shared-memory multithreaded baseline as computational resources scale.

Methodology. The comparison focuses on the Adaptive Random Forest algorithm, as it is the primary ensemble method in **MOA** supporting native multi-threading. We evaluate

Dataset	2 Core	4 Core	8 Core	16 Core	32 Core
Airlines	1.92	3.45	5.10	8.33	11.59
Elec	1.94	3.67	5.89	8.26	11.04
Covtype	2.00	4.02	6.55	9.60	15.81
KDD99	1.77	3.42	5.51	8.05	9.87
Internet ads	1.96	3.23	3.91	5.26	5.95
Nomao	1.94	3.93	6.50	8.90	13.20
Spam Corpus	1.80	2.56	2.75	2.88	2.92
LED(a)	1.92	3.52	5.42	7.37	10.39
LED(g)	1.92	3.50	5.48	7.52	10.49
AGR(a)	2.00	3.86	6.12	9.02	14.98
AGR(g)	2.02	3.88	6.07	9.02	14.98
RBF(m)	2.01	3.96	6.53	10.25	17.66
RBF(f)	2.02	3.96	6.47	10.06	16.78

Table 4.14: Speedup Ratios for AMF, 20-tree ensemble



(a) Speed up of RBF(m)

(b) Speed up of Spam Corpus

Figure 4.3: Speed up behaviors: notable results.

efficiency on two representative datasets: $RBF(m)$, which previously demonstrated the highest efficiency for our system, and **Internet ads**, which serves as a proxy for less efficient workloads after **Spam Corpus** failed to execute in the MOA environment.

Analysis of Results. The results in Table 4.15 reveal a significant performance gap. On the $RBF(m)$ dataset, Renoir achieves 40.3% efficiency at 32 cores, more than triple the 12.32% recorded for MOA. This suggests that MOA’s centralized approach suffers from severe synchronization bottlenecks and resource contention as the thread count increases. Even in the **Internet ads** scenario, Renoir maintained a lead, showing 28.6% efficiency against MOA’s 18.59% at 32 cores.

Furthermore, the failure of MOA to complete the execution on the **Spam Corpus** dataset (as previously discussed in Section 4.2.1) highlights a critical robustness advantage.

Dataset	Renoir 16 core	Renoir 32 core	MOA 16 core	MOA 32 core
Internet ads	45.1	28.6	41.66	18.59
Spam Corpus	35.1	16.9	<i>Crash</i>	<i>Crash</i>
RBF(m)	51.8	40.3	26.30	12.22

Table 4.15: Scalability efficiency of ARF (%)

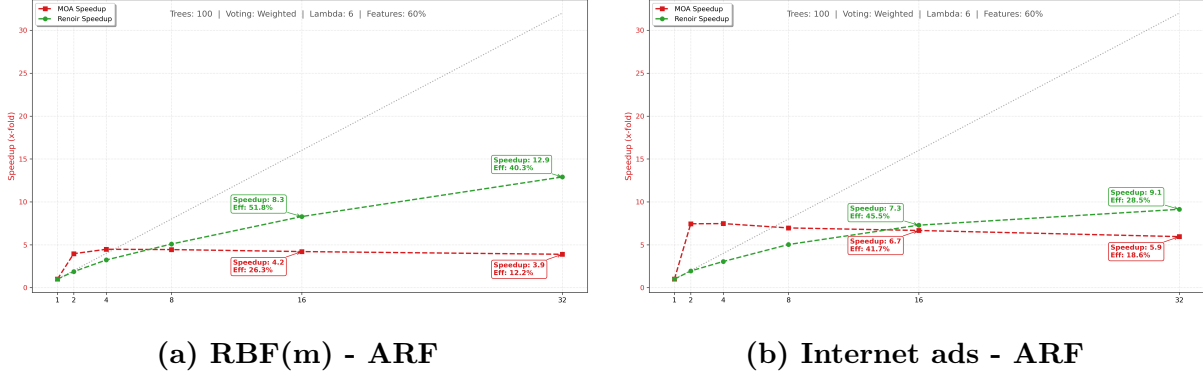


Figure 4.4: Speedup comparison between MOA and Renoir: notable results.

The choice of these datasets allows for an interesting observation regarding the performance boundaries of both systems. We observe an inverse performance trend: **RBF(m)**, which yields the highest efficiency in Renoir, results in the lowest for MOA.

This inversion suggests a fundamental difference in architecture-specific **bottlenecks**. MOA suffers from centralized memory contention and thread synchronization overhead in computationally intensive tasks like **RBF(m)**, where the complexity of the trees likely increases heap pressure and lock contention. In contrast, Renoir thrives by distributing these heavy tasks across independent units.

Conversely, in lighter workloads like **Internet ads**, the communication and serialization overhead inherent in distributed systems becomes more prominent for Renoir. Meanwhile, MOA’s shared-memory approach is at its most effective here because the low computational intensity reduces the frequency of synchronization conflicts, although it still remains less efficient than Renoir overall at higher core counts.

The scalability trends are further illustrated in Figure 4.4. While the speedup for Renoir follows a consistent, though sub-linear, growth as discussed previously, MOA exhibits anomalous behavior. In both Figures 4.4a and 4.4b, MOA shows a super-linear speedup when transitioning from one to two cores (approximately 7x), which may be attributed to internal resource management or cache-locality benefits specific to its multithreaded implementation. However, this performance gains immediately plateaus; adding further

threads does not yield higher throughput and, in several instances, results in a slight performance degradation. This saturation confirms that MOA’s parallel implementation is unable to effectively exploit high-degree parallelism.

4.2.5. Comparison with Distributed VHT

This section evaluates the proposed distributed ensemble methods against the Renoir implementation of the Vertical Hoeffding Tree (VHT). Both systems are implemented within the same distributed framework, allowing for a controlled comparison of their respective architectural trade-offs.

Vertical Hoeffding Tree (VHT) Algorithm

The Vertical Hoeffding Tree (VHT) is a distributed, data-parallel adaptation of the VFDT designed to handle high-velocity data streams by scaling across multiple nodes. While standard Hoeffding Trees are efficient, a single-node execution is often bottlenecked by the memory required to store sufficient statistics for split evaluation.

VHT addresses this through **vertical parallelism**, where the data matrix is partitioned by columns (features) rather than rows (instances). In this architecture, the workload is split between two main components:

- **Model Aggregator:** Maintains the global tree structure and performs instance routing. It coordinates the training process by identifying the destination leaf for incoming instances and managing the final split decisions using the Hoeffding bound.
- **Local Statistics Workers:** Distributed nodes that store and update the sufficient statistics for specific subsets of features. Because statistics for a given feature are owned by a single worker, split scores can be computed locally without the need for global state replication or heavy synchronization.

The training workflow is event-driven. When a labeled instance arrives, the Aggregator routes it to a leaf and emits *feature-wise* messages to the corresponding Local Statistics workers. Periodically, the Aggregator triggers a *compute* request; workers return their local best split candidates, and the Aggregator performs a split if the difference in gain $\Delta\bar{G}$ satisfies the Hoeffding condition.

Distributed VHT Algorithm

To transform the conceptual VHT into a high-performance distributed system, several architectural extensions were implemented:

- **Replaying Mechanism:** A *Per-Leaf Ordering Stage* ensures consistent epoch boundaries under sparsity. To mitigate accuracy loss from "in-flight" instances, an *Instance Replaying* mechanism buffers instances in per-leaf queues and replays them under the updated model after a split occurs.
- **Resource Awareness:** To ensure a bounded state, the system enforces a *maximum tree depth*. It also implements a coordinated *memory reclamation* protocol using Drop control messages to prune obsolete distributed statistics.
- **High-Throughput Parallelism:** *Routing Replication* parallelizes instance processing across multiple cores on the main host. The concurrency model was optimized by replacing global locks with `RwLock` and `AtomicUsize`, allowing concurrent routing while sacrificing absolute consistency for massive throughput gains.

Performance analysis

Goal. The primary goal of this comparison is to quantify the performance gap between a single high-throughput distributed tree and an ensemble of distributed learners. We evaluate these models across two primary dimensions: predictive accuracy and system throughput.

Methodology. For this comparison, we focus exclusively on the **real-world datasets** described in Section 4.1.3. We have explicitly omitted experimental runs on the idealized datasets used in the original VHT work (URT and SGZ). In environments characterized by non-redundant feature spaces and an absence of concept drift, a distributed ensemble is at a structural disadvantage; the lack of redundancy minimizes the statistical benefits of learner diversity, while the computational overhead of maintaining multiple distributed models remains high. Similarly, synthetic datasets were excluded because the current distributed VHT implementation lacks an integrated drift detection mechanism. Including them would result in an uninformative degradation of accuracy that reflects a missing feature rather than an architectural flaw.

The experimental configuration maintains consistency with the parameters detailed in Section 4.2.1 (summarized in Table 4.2), specifically utilizing an ensemble size of 100 trees, and the maximum available cores. Furthermore, all shared hyperparameters, such as Hoeffding bound constants and histogram binning resolutions, have been synchronized across both implementations. This configuration represents a **real-world scenario** where the user leverages all available parallelism: for VHT, this resource allocation maximizes throughput via vertical partitioning, whereas for the ensemble, it maximizes predictive performance through model diversity.

To ensure a fair comparison, the **replay mechanism** is enabled in the distributed VHT. In this high-velocity distributed streams, instances are often lost or processed by stale model states during the asynchronous split process. Enabling the replay mechanism ensures that these instances are buffered and re-entered into the pipeline once the model updates. This is crucial for a fair evaluation because it forces VHT to account for (almost) every instance in the stream, consistent with how the ensemble methods operate, preventing an artificial inflation of throughput at the expense of discarded data.

Note that empirical comparisons are performed specifically against Adaptive Random Forest (ARF), as preliminary tests indicated that SRP yields nearly identical performance characteristics in this context.

Analysis of Results. The results for the real-world datasets are presented in Table 4.16. The data reveals a consistent trend: ensemble methods provide superior predictive accuracy at the cost of a significant reduction in throughput.

The accuracy gap is particularly evident in datasets like *Covtype* and *Nomao*, where the presence of concept drift severely impacts the performance of the non-adaptive VHT. While an implementation of drift detection within VHT would likely narrow this gap, the ensemble’s ability to maintain a robust decision boundary through weighted voting suggests it would remain the more accurate choice in volatile environments.

The throughput analysis, however, shows a substantial *speedup* in favor of VHT. Interestingly, this performance gap is highly sensitive to the feature dimensionality of the dataset:

- **High-Dimensional Streams:** In datasets with a high number of features (e.g., *Internet ads* and *Spam Corpus*), the throughput gap narrows significantly. This suggests a potential scalability bottleneck in VHT’s vertical partitioning when dealing with very large feature sets. In contrast, the ensemble’s *feature patching* mechanism limits the number of features per learner, mitigating the computational load per tree and making the ensemble relatively more efficient as dimensionality grows.
- **Low-Dimensional Streams:** In datasets with fewer features, the overhead of managing 100 separate distributed learners results in a much lower speedup relative to the streamlined, single-tree architecture of VHT.

Conclusion. The deployment decision should be driven by the specific use case:

- **Accuracy-Critical Applications:** In domains where predictive precision and drift resilience are paramount (e.g., fraud detection or security filtering), the ensemble

Dataset	Accuracy ARF	Accuracy VHT	Throughput ARF	Throughput VHT	Speedup
Airlines	67.36 ± 0	60.17 ± 0.67	$171,367 \pm 2,729$	$2,427,299 \pm 95,328$	0.07
Elec	89.27 ± 0.02	57.54 ± 0.00	$159,786 \pm 4,926$	$982,388 \pm 39,867$	0.16
Covtype	94.99 ± 0	58.81 ± 1.05	$64,560 \pm 2,128$	$789,962 \pm 11,242$	0.08
Kdd99	99.99 ± 0	97.97 ± 0.36	$79,746 \pm 3,731$	$684,686 \pm 20,321$	0.12
Internet ads	99.82 ± 0	85.63 ± 0.14	$12,942 \pm 491$	$20,716 \pm 2,938$	0.62
Nomao	96.9 ± 0.01	71.62 ± 0.13	$45,880 \pm 1,127$	$260,142 \pm 6,409$	0.18
Spam Corpus	97.38 ± 0.02	79.69 ± 1.02	857 ± 8	$1,079 \pm 107$	0.79

Table 4.16: VHT comparison with ARF using 100 base models and 32 cores

method is the superior choice, especially since the throughput penalty becomes less severe as the complexity of the data increases.

- **Throughput-Critical Applications:** In scenarios requiring massive data ingestion speeds with limited computational resources, the streamlined architecture of VHT remains the most efficient solution.

In summary, as feature dimensionality increases, the ensemble approach provides a compelling middle ground, offering significantly higher accuracy with a throughput cost that may, in many real-world scenarios, be considered negligible.

Baseline Framework Transition Analysis

Goal. This analysis aims to quantify the relative efficiency gains achieved when transitioning from the MOA (centralized) to the Renoir (distributed) environment. Specifically, we compare the throughput escalation of the proposed ensemble methods against the throughput escalation observed in the distributed VHT implementation. This allows us to evaluate whether the distributed optimizations in this work offer a more significant framework-level improvement than those in prior distributed decision tree literature [11].

Methodology. The evaluation focuses on Adaptive Random Forest (ARF) as the representative ensemble method, given its characteristic performance profile. We utilize two synthetic datasets to serve as performance benchmarks:

- **AGR(a):** Selected because it previously yielded the highest framework-to-framework speedup, serving as a best-case scenario for distributed optimization.
- **LED(a):** Utilized as a proxy for workloads with higher computational density per instance, replacing *Spam Corpus*, which was unable to execute within the MOA environment’s memory constraints.

By comparing the speedup factor of VHT against that of ARF, we can isolate the impact

Dataset	Throughput MOA VHT	Throughput Renoir VHT	Speedup VHT	Speedup ARF
AGR(a)	493,254	1,550,313	3.14	21.82
LED(a)	296,240	590,503	1.99	6.01

Table 4.17: Comparison of throughput speedup factors when transitioning from MOA to Renoir for VHT and ARF.

of the framework’s distributed architecture on different model complexities.

Analysis of Results. The comparative speedup metrics are detailed in Table 4.17.

The results indicate a substantial disparity in framework efficiency gains. While the transition from MOA to Renoir provides VHT with a notable throughput increase (ranging from 1.99x to 3.14x), the acceleration factor for ARF is significantly higher, reaching a 21.82x improvement on the *AGR(a)* dataset. This suggests that the distributed optimizations implemented in this work are particularly effective for ensemble structures. While a single distributed tree like VHT eventually faces overheads related to vertical synchronization and attribute splitting, the ensemble methods leverage the Renoir framework to parallelize the independent processing of multiple base models, resulting in a more scalable architecture. Consequently, the performance leap demonstrated in this work represents a major advancement in distributed stream mining, outperforming the relative efficiency gains achieved by previous state-of-the-art distributed transitions.

4.2.6. Hyperparameter Influence

Goal. In this section, we provide both a theoretical and practical analysis of how the hyperparameters of the selected algorithms influence performance, specifically focusing on the trade-off between throughput and accuracy.

- **Ensemble size(s):** Across all ensemble methods, the number of base learners is the most critical factor for accuracy. Traditionally, this parameter scales the computational load linearly; however, in our work, the impact on throughput is mitigated by our distributed implementation. As further analyzed in Section 4.2.4, parallelization across multiple cores allows for larger ensembles with significantly reduced overhead compared to sequential execution.
- **Voting mechanism:** It plays a vital role, as discussed in Section 2.1.1. While more sophisticated aggregation methods (e.g., weighted majority voting) require additional logic, the overhead is typically negligible compared to the time consumed by the base learners’ training and prediction phases. Consequently, it is generally

preferable to employ a more refined mechanism to maximize accuracy.

Hoeffding-Tree Based Ensembles

The Hoeffding-tree based ensembles, namely SRP and ARF, are particularly sensitive to the following parameters:

Ensemble Configuration

- **Feature Patch Size (m):** This determines the number of features considered for each split. A smaller patch size significantly increases throughput due to a reduced search space but may result in lower accuracy. However, this is not universally true; in high-noise datasets, reducing the patch size can improve robustness and accuracy by preventing the model from overfitting to noisy features.
- **Lambda (λ):** In methods utilizing Online Bagging, λ represents the parameter of the Poisson distribution. Higher values increase the diversity between trees, which can improve accuracy but inversely reduces throughput. This parameter requires careful tuning, as excessively high values may lead to overfitting.

Base Learner (Single Tree) Configuration

- **Drift Detection:** In scenarios where the data stream is known to be stationary, disabling drift and warning detectors (e.g., ADWIN) improves both throughput and accuracy. This prevents the computational cost of training background learners and avoids unnecessary tree resets due to false positives during "warm-up" periods. However, drift detection can be beneficial even without true concept drift, as it may mitigate the impact of sub-optimal feature subspaces selected by individual nodes.
- **Grace Period (n_{min}):** This defines the number of instances observed between split attempts. A larger grace period accelerates processing by reducing the frequency of expensive split calculations, though it may result in a less granular (and potentially less accurate) tree structure.
- **Hoeffding Parameters (δ and τ):** The Hoeffding bound (δ) and the tie-breaking threshold (τ) dictate the strictness of the split criteria, influencing how quickly the tree grows.
- **Numeric Estimator:** The choice between *Gaussian* or *Histogram* estimators depends on the dataset. Histograms are more flexible for complex distributions but often require more memory and computation than the Gaussian approach.

Practical Analysis and Trade-off Evaluation

To validate these theoretical observations, we conducted experiments on two representative datasets: **Internet ads**, a real-world dataset characterized by high feature dimensionality, and **LED(g)**, a synthetic dataset with a high instance count. We varied three primary parameters: the number of trees s (10 or 100), the feature patch size m (60% or $\sqrt{\text{features}}$), and the λ parameter (1 or 6).

As shown in Table 4.18, the highest accuracy is consistently achieved by the most complex configuration ($s = 100$, $m = 60\%$, $\lambda = 6$), while the highest throughput is achieved by the most minimal configuration ($s = 10$, $m = \sqrt{\text{features}}$, $\lambda = 1$).

To evaluate the balance between model efficacy and computational efficiency, we define a **Normalized Efficiency Score** (S):

$$S_i = w \cdot \left(\frac{A_i}{\max(A)} \right) + (1 - w) \cdot \left(\frac{T_i}{\max(T)} \right) \quad (4.8)$$

where A and T represent Accuracy and Throughput respectively, and w is a weighting factor (e.g., $w = 0.75$) used to prioritize predictive performance over speed. This specific value reflects a standard preference in production systems where model quality is the primary objective, but computational costs cannot be ignored. Notably, the relative ranking of the configurations is highly robust to this choice; even if w is reduced to 0.5 (equal weight) or lowered significantly further, the optimal configuration remains unchanged due to the steep throughput penalties incurred by the more complex models.

The experimental results reveal distinct sensitivities for each parameter across accuracy and throughput. By aggregating the performance deltas across both datasets, we can quantify the average impact of shifting each variable from its lower to its higher bound.

Ensemble Size (s) Increasing the ensemble size from 10 to 100 learners represents the most significant computational bottleneck. While it provides a marginal average accuracy gain of approximately +0.24%, it incurs a severe throughput penalty, reducing processing speed by an average of 315,892 instances per second. Consequently, despite the 75% weight assigned to accuracy, the efficiency score S drops by an average of 11.55%. This indicates that for these specific datasets, the redundant learners contribute more to computational overhead than to predictive robustness.

Feature Patch Size (m) The transition from sparse feature patches ($\approx 2.6\%$ for Internet ads and 20.8% for LED(g)) to large patches ($\approx 60\%$) results in a moderate accuracy

increase of +0.33%. However, this gain is offset by a throughput decrease of approximately 238,994 instances per second. The resulting 7.14% average decline in S suggests that the models are highly efficient at extracting signal from a reduced feature space, and the additional dimensions provided by larger patches yield diminishing returns.

Poisson Lambda (λ) The resampling intensity, controlled by λ , proves to be the most efficient lever for performance optimization. Increasing λ from 1 to 6 yields the highest average accuracy boost (+0.49%) with the least significant impact on throughput (a reduction of 110,674 instances per second). Accordingly, λ has the smallest negative impact on the efficiency score S (−2.66%). This suggests that increasing the training weight per instance via Poisson resampling is computationally "cheaper" than increasing the number of trees or the feature count per node.

Optimal Configurations As highlighted in Table 4.18, the configuration achieving the highest S value (99.15% for **Internet ads** and 98.97% for **LED(g)**) consistently utilizes a small ensemble size ($s = 10$) and minimal feature patches. This confirms that in high-velocity streaming environments where $w = 0.75$, the optimal strategy is to deploy lightweight learners and use λ as the primary mechanism to tune for predictive accuracy, rather than scaling the physical size of the ensemble.

It is important to qualify these throughput results within the context of the underlying hardware architecture. All configurations were executed on a 32-core processor. In the $s = 10$ configuration, the system is under-utilized, with only one tree per core and a significant portion of the computational resources remaining idle. Conversely, the $s = 100$ configuration achieves a higher density of approximately 3.125 trees per core, leading to resource contention and increased overhead from thread synchronization and context switching. While this disparity may bias the throughput comparison toward smaller ensembles, the analysis remains valid for real-world deployments.

Furthermore, it should be noted that the "efficiency-first" approach dictated by S is not universally applicable. In certain critical domains where even a marginal increase in predictive error incurs significant costs or risks, accuracy becomes the absolute priority ($w \rightarrow 1.0$). In such cases, a bigger ensemble size remains the preferred choice, as it provides the global maximum for accuracy regardless of the resulting throughput degradation. This highlights the necessity of aligning hyperparameter selection with the specific cost-benefit profile of the target application.

Dataset	s	m	λ	A (%)	T (inst/s)	S (%)
Internet ads	10	60%	1	98.66	40,508	97.72
	10	60%	6	99.81	39,965	98.27
	10	2.6%	1	98.69	42,930	99.15
	10	2.6%	6	99.33	41,423	98.75
	100	60%	1	98.81	17,170	84.24
	100	60%	6	99.82	12,913	82.52
	100	2.6%	1	98.66	35,542	94.83
	100	2.6%	6	99.33	34,696	94.84
LED(g)	10	62.5%	1	73.02	701,583	91.02
	10	62.5%	6	73.10	280,667	81.40
	10	20.8%	1	72.17	1,084,280	98.97
	10	20.8%	6	72.51	777,702	92.25
	100	62.5%	1	73.13	89,927	77.03
	100	62.5%	6	73.17	38,390	75.89
	100	20.8%	1	73.15	176,247	79.04
	100	20.8%	6	73.16	77,041	76.77

Table 4.18: Performance metrics (Accuracy and Throughput) across different hyperparameter configurations for Internet ads and LED(g) - ARF

Aggregated Mondrian Forest (AMF)

The AMF is an ensemble method based on Mondrian Trees, utilizing a Restricted Mondrian Process. The key parameters governing the behavior of a single Mondrian Tree within the forest are:

- **Step:** This parameter determines the step size (or learning rate) for the online weight updates of the nodes. It controls how strictly the node weights are penalized based on the probability of predicting the correct class. By tuning the step parameter, one can dictate how quickly the AMF adapts to new data, which directly controls its responsiveness to concept drift.
- **Dirichlet:** This parameter sets the concentration of the Dirichlet prior used to smooth the probability estimates at the leaf nodes. A larger Dirichlet value increases smoothing, meaning the model requires a higher volume of samples to make confident predictions based on the data. Conversely, a smaller Dirichlet value reduces smoothing, allowing the model to base its predictions strictly on the observed empirical class counts even when the sample size is very small.

- **Split Pure:** This boolean flag determines whether a pure node (a node containing samples from only a single class) should continue to split. Enabling this flag increases the maximum depth of the tree, which prepares the structure for future unseen data in an online setting. However, this comes at the cost of penalizing memory and throughput, and depending on the data distribution, it can lead to either improved or degraded overall accuracy.

Practical Analysis and Trade-off Evaluation

Unlike the ARF and SRP analysis, where three parameters (s, m, λ) were varied simultaneously, the AMF evaluation focuses exclusively on ensemble size s . This methodological choice is motivated by empirical observation: preliminary experiments showed that varying *step* and *Dirichlet* within their practical ranges produces negligible differences in both accuracy and throughput across the tested datasets. Ensemble size therefore remains the dominant and most actionable parameter, consistent with the findings in Section 4.2.6 for the other ensemble methods.

For the same reason, the Normalized Efficiency Score S defined earlier is not directly applied here. Since the non-ensemble hyperparameters are held fixed, the trade-off reduces to a two-variable comparison between accuracy and throughput as a function of s alone; relative throughput (normalized to the single-tree baseline) conveys this relationship more directly than a weighted composite score.

The datasets were also selected to reflect representativeness criteria suited to AMF's specific characteristics. Among the real-world datasets, `covtypeNorm` was chosen because it presents a well-balanced ratio between feature dimensionality and the total volume of instances, making it highly representative of standard streaming workloads. From the synthetic datasets, `RBf(f)` was selected due to its low baseline accuracy, which amplifies the marginal gains from increasing ensemble size and makes the diminishing-returns effect clearly observable.

As shown in Table 4.19, increasing the number of trees incurs a notable cost to relative throughput. From 10 to 15 trees, the average cost ranges between 1% and 2% of throughput per added tree; however, beyond 15 trees, the degradation rate accelerates to approximately 2.5% per tree. Consequently, in strict streaming environments where maximizing throughput and minimizing latency are the primary operational constraints, restricting the ensemble to 15 trees provides an optimal balance between accuracy and computational efficiency.

Conversely, while increasing the number of trees yields a direct benefit in predictive accu-

racy, this improvement strictly follows a law of diminishing returns. The severity of these diminishing returns is highly dataset-dependent. For example, with the `covtypeNorm` dataset, increasing the ensemble size beyond 50 trees offers a poor trade-off, as accuracy plateaus while throughput continues to plummet. In contrast, the `RBF(f)` dataset exhibits more sustained accuracy improvements at larger ensemble sizes, justifying the addition of more trees if resources permit. Ultimately, because the AMF algorithm possesses an upper bound on achievable accuracy, the final ensemble size must be strategically decided based on the specific application’s tolerance for throughput degradation versus its need for predictive precision.

Dataset	Trees (<i>s</i>)	<i>A</i> (%)	<i>A</i> Gain	Rel. <i>T</i> (%)
covtypeNorm	10	92.49	+0.00	100.00
	15	92.77	+0.28	93.45
	20	92.88	+0.39	75.31
	30	93.09	+0.60	52.72
	40	93.15	+0.66	32.73
	50	93.21	+0.72	28.96
	60	93.24	+0.75	24.88
	70	93.29	+0.80	18.91
	80	93.29	+0.80	17.60
RBF(f)	10	44.71	+0.00	100.00
	15	46.69	+1.98	90.22
	20	47.96	+3.25	76.11
	30	49.49	+4.78	55.49
	40	50.42	+5.71	31.90
	50	51.05	+6.34	28.30
	60	51.51	+6.80	23.90
	70	51.84	+7.13	17.50
	80	52.11	+7.40	16.04

Table 4.19: Trade-off Evaluation: Performance vs. Relative Gains Across Ensemble Sizes - AMF

4.3. Discussion

This section synthesizes the findings from the experimental evaluation presented in Section 4.2 and provides direct answers to the Research Questions stated at the beginning of this chapter.

Q1: How does the proposed ensemble approach compare to the baseline in terms of predictive accuracy and computational efficiency? The experiments in Section 4.2.1, Section 4.2.2 and Section 4.2.3 jointly answer this question. With respect to predictive accuracy, the results in Table 4.3, Table 4.4 and Table 4.9, plus the TOST formal verifications, demonstrate that the Renoir implementations of ARF, SRP and AMF, achieve accuracy parity with MOA across all tested datasets, with deviations well within the expected range, confirming functional correctness.

In terms of baseline computational efficiency, the single-core throughput results in Table 4.5 reveal speedup factors ranging from $1.97\times$ to $35.73\times$ over MOA, with the largest gains observed on high-dimensional datasets such as **Spam Corpus**. These gains are primarily explained by Rust’s zero-cost abstractions and efficient memory management compared to MOA’s JVM-based heap allocations. Likewise, the Rust-based AMF implementation delivers speedups of up to $18.24\times$ over Onelearn on low-dimensional datasets (Table 4.10), primarily through bounds check elision and efficient slice computations. Taken together, these results confirm that the proposed implementation maintains the predictive integrity of the original algorithms while offering substantial computational advantages in a sequential setting.

When scaling the workload to a multi-core configuration (100 trees across 32 cores), the results in Table 4.6 expose a critical architectural limitation of the baseline, which can be characterized as a *concurrency ceiling*. Even though the computational load per thread decreases when moving from the single-core baseline to the 32-core setup, MOA’s relative performance degrades due to lock contention within its shared-memory thread pool and severe JVM Garbage Collection overhead across 32 active threads. Conversely, Renoir’s inherently decoupled dataflow model bypasses these centralized bottlenecks, allowing its relative speedup to expand significantly (e.g., rising to $21.82\times$ on the **AGR(a)** dataset). Furthermore, the multi-core evaluation underlines a stark contrast in robustness; while MOA suffers fatal out-of-memory crashes or timeouts when handling high-dimensional streams like **Spam Corpus** under heavy parallel execution, our implementation successfully processes the stream, demonstrating superior memory stability under high concurrency.

Finally, the remote execution experiments detailed in Table 4.11 address the framework’s behavior in a multi-node environment, illustrating the explicit trade-off between computational density and communication overhead. The findings reveal that execution efficiency is inversely correlated with dataset dimensionality. For low-dimensional streams (such as **Airlines** or the synthetic **LED** suites), Renoir preserves over 90% of its local parallel execution efficiency, as the processing demands of the ensemble dominate the minimal data transfer costs. However, in hyper-dimensional scenarios like **Internet**

ads and Spam Corpus, efficiency falls to 35.03% and 26.99% respectively, indicating that the serialization of massive feature vectors and network coordination over SSH becomes the primary system bottleneck. Crucially, even when burdened with this communication penalty, Renoir’s absolute distributed throughput remains orders of magnitude higher than MOA’s centralized baseline, verifying that the distributed dataflow model remains highly advantageous for cluster deployments.

Q2: To what extent does the Renoir implementation improve computational throughput and scalability relative to a sequential execution? The scalability analysis in Section 4.2.4 directly addresses this question. The results in Table 4.13 show that the Renoir implementation of ARF and SRP consistently scales throughput as the number of cores increases from 1 to 32, reaching parallel efficiencies close to 50% on physical cores for most datasets and ensemble methods. Regarding AMF, the analysis demonstrates a distinct speedup behavior shaped by strict algorithmic constraints. Although it achieves peak speedups of up to 17.66 on the RBF(m) dataset (Table 4.14), its scalability plateaus earlier than the other ensemble methods because the 20-tree limit inherently leaves execution units idle at higher thread counts, and high-dimensional streams like Spam Corpus shift the bottleneck to memory bandwidth. The decentralized aggregation strategy via `group_by(instance_id)`, described in Section 3.2, is identified as a key enabler of this scaling behavior, preventing the many-to-one bottleneck typical of centralized voting stages.

The system’s efficiency does degrade when transitioning from 16 physical to 32 logical threads (an approximate 10% drop), consistent with the shared execution resources of Simultaneous MultiThreading (SMT). An additional boundary case is the Spam Corpus dataset, where the extreme feature dimensionality (39,917 features) causes memory bus contention and cache thrashing, resulting in a throughput regression at 32 threads. Regarding speedup, however, a marginal increase is still observed when transitioning from 16 to 32 threads. This highlights that while Renoir effectively converts additional hardware into processing throughput for the vast majority of workloads, the memory subsystem represents the ultimate scalability ceiling for ultra-high-dimensional data.

The comparison with MOA’s parallel ARF in Section 4.2.4 and Table 4.15 further contextualizes this result: on the RBF(m) dataset, Renoir achieves 40.3% efficiency at 32 cores against MOA’s 12.22%, confirming that the dataflow-based distributed model scales significantly more gracefully than the centralized shared-memory alternative.

Q3: How does the proposed ensemble architecture perform in comparison to the state-of-the-art distributed VHT baseline in terms of predictive performance and raw throughput? Section 4.2.5 addresses this question through a controlled comparison of 100-tree ARF against the distributed VHT on real-world datasets (Table 4.16). The results reveal a clear architectural trade-off: the ensemble methods consistently achieve substantially higher predictive accuracy, with the gap being most pronounced on datasets affected by concept drift. For instance, on *Covtype* and *Nomao*, VHT accuracy falls to 58.81% and 71.62% respectively, while ARF achieves 94.99% and 96.9%. This is largely attributable to VHT’s lack of an integrated drift detection mechanism, which leaves it unable to adapt to non-stationary distributions.

The throughput analysis, however, favours VHT in low-dimensional settings, where its streamlined single-tree architecture processes instances at significantly higher rates. This gap narrows considerably as feature dimensionality increases: on *Internet ads* and *Spam Corpus*, VHT’s throughput advantage is reduced to a factor of 0.62 and 0.79 respectively, due to scalability bottlenecks in vertical feature partitioning under high dimensionality. The ensemble’s *feature patching* mechanism, which limits the active feature count per learner, provides a natural mitigation to this bottleneck. Consequently, the two architectures are complementary: VHT is the preferred choice for throughput-critical tasks on stationary, low-dimensional streams, while the ensemble methods are superior for accuracy-critical applications in evolving, high-dimensional environments.

Q4: How does the throughput speedup achieved by the proposed ensemble methods, when transitioning from a centralized to a distributed framework, compare to the speedup observed in existing distributed decision tree implementations? The baseline framework transition analysis in Section 4.2.5 (Table 4.17) directly addresses this question by comparing the MOA-to-Renoir speedup factors for ARF and VHT on the same datasets. The data shows that while VHT’s transition from MOA to Renoir yields a throughput increase of $1.99\times$ to $3.14\times$, the ARF ensemble achieves a substantially higher improvement of $6.01\times$ on *LED(a)* and $21.82\times$ on *AGR(a)*.

This disparity highlights a key architectural insight: ensemble methods are intrinsically better suited to the distributed dataflow paradigm than single-tree approaches. The inherent independence of the base learners allows Renoir to parallelize their execution with minimal synchronization overhead, whereas a single distributed tree such as VHT eventually encounters bottlenecks associated with vertical feature partitioning and coordinated split decisions. As a result, the framework-level gains achieved in this work significantly surpass those of prior distributed decision tree literature, establishing the proposed ap-

proach as a more impactful contribution to the field of distributed stream mining.

Q5: To what extent do hyperparameter configurations influence the model’s stability, and what are the optimal settings for diverse data stream conditions?

The hyperparameter analysis in Section 4.2.6 provides a structured answer to this question by evaluating the sensitivity of SRP and ARF to ensemble size (s), feature patch size (m), and Poisson lambda (λ) across two representative datasets. For the AMF implementation, because ensemble size is the primary determinant of predictive accuracy, the analysis focuses specifically on the number of trees.

The results in Table 4.18 indicate that the ensemble size is the dominant factor affecting throughput, with a transition from $s = 10$ to $s = 100$ reducing processing speed by an average of 315,892 instances per second while yielding only a marginal +0.24% accuracy gain. The feature patch size proves to be a moderate lever: reducing m to $\sqrt{\text{features}}$ preserves most of the accuracy while substantially improving throughput, as smaller feature subsets reduce the computational cost of split evaluation. In contrast, the Poisson resampling parameter λ offers the most favourable accuracy-to-cost trade-off, delivering the highest average accuracy improvement (+0.49%) with the least throughput penalty.

These observations lead to a practical recommendation: in high-velocity streaming scenarios where the efficiency score S is employed as the optimization criterion (with $w = 0.75$), the optimal configuration uses a small ensemble with minimal feature patches and an elevated λ to compensate for learner diversity. However, in accuracy-critical domains, a larger ensemble remains the global maximum for predictive performance, underscoring the importance of aligning hyperparameter selection with the cost-benefit profile of the target application.

Regarding AMF, the empirical analysis reveals a unique hyperparameter sensitivity profile. Preliminary evaluations demonstrated that varying internal tree parameters, such as the learning *step* and *Dirichlet* smoothing, produced negligible differences in both accuracy and throughput across the tested datasets. Consequently, the ensemble size (s) emerges as the sole actionable parameter. The trade-offs detailed in Table 4.19 demonstrate that increasing the number of trees yields a steady improvement in accuracy at the direct expense of computational throughput. Specifically, adding trees up to an ensemble size of 15 incurs a throughput penalty of 1% to 2% per tree, a degradation rate that accelerates to approximately 2.5% per tree beyond this threshold. Furthermore, the accuracy gains follow a strict law of diminishing returns that is highly dataset-dependent; for instance, accuracy on the `covtypeNorm` stream plateaus completely after 50 trees, whereas the more complex `RBf(f)` stream sustains marginal improvements at larger sizes.

5 | Related work

This chapter provides an overview of the state-of-the-art concerning current stream machine learning frameworks. We first discuss how traditional machine learning approaches attempt to resolve the inherent conflicts associated with continuous data streams, followed by an examination of true stream learning frameworks. Finally, we outline the existing directions for scaling these systems through vertical and horizontal parallelism, and position the contribution of this thesis within this landscape.

Traditional machine learning frameworks often rely on a technique called micro-batching (or mini-batching), which consists of collecting fixed-size batches of data and processing them sequentially to simulate stream learning. Frameworks such as **Apache Spark MLlib** [27] frequently utilize this approach. While subsequent distributed stream processing engines like **Apache Storm** [35] and **Apache Flink** [9] introduced true continuous event-at-a-time processing models, their native machine learning libraries still largely rely on batch-oriented approximations. Ultimately, the traditional batch-processing paradigm fundamentally relies on buffering data, which violates the strict single-pass, bounded-memory constraints required for true stream learning.

The current landscape of true stream machine learning [7, 21] is dominated by a few key frameworks that natively process data one instance at a time. However, these tools are generally constrained by centralized, single-node execution models. **Massive Online Analysis (MOA)** [6] serves as the de facto standard open-source framework for mining evolving data streams, offering a robust, Java-based platform with an extensive collection of machine learning algorithms for classification, regression, and clustering. However, its JVM-based execution introduces non-trivial memory overhead, and its multi-threading support is limited to a handful of specific algorithms rather than exposed as a general parallel primitive. Alternatively, **River** [29] is a Python library designed to be the most user-friendly tool for performing machine learning on streaming data. Born from the evolution of previous foundational libraries such as **scikit-multiflow** [28], River is continually evolving, regularly incorporating state-of-the-art algorithms, and supports dataset preprocessing alongside pipeline creation. Its single-threaded design, however, means throughput

is fundamentally bounded by a single CPU core, making it unsuitable for high-velocity streams at scale. A related Python library is **Onelearn** [32], which provides efficient implementations of Mondrian-based online learners, including the Aggregated Mondrian Forest (AMF). While Onelearn offers a clean and accessible interface for online Mondrian forests, it similarly operates in a single-threaded, centralized fashion and is not designed for distributed or high-throughput deployments.

A significant limitation of these frameworks is their inability to fully exploit distributed execution models. River operates in a strictly single-threaded environment, inherently constrained by Python’s execution architecture. While MOA provides limited support for multi-threading in specific algorithms, it does not extend to a general distributed architecture. These constraints become particularly acute for ensemble methods, which maintain hundreds of independently trained base learners: the sequential update bottleneck grows linearly with ensemble size, making centralized execution fundamentally incompatible with high-throughput streaming requirements.

To overcome these computational bottlenecks, researchers have explored different distributed architectures. Early efforts led to the development of frameworks like **Scalable Advanced Massive Online Analysis (SAMOA)** [30], which attempted to distribute standard SML algorithms across engines like Flink but suffered from severe centralized coordination bottlenecks, as the aggregation of model updates still required passing through a single coordinator node. To address these inefficiencies, subsequent research explored vertical parallelism. A notable streaming example is the Vertical Hoeffding Tree (VHT) [24], a distributed adaptation of the Hoeffding Tree that scales by evaluating feature splits across separate nodes. While VHT demonstrates that distributed streaming learning is feasible, it is restricted to a single base algorithm and does not generalize to ensemble methods, leaving horizontal parallelism of ensemble classifiers as an open problem.

Building upon the need for robust distributed scaling, the core contribution of this thesis is centered on horizontal parallelism. Unlike JVM-based runtimes such as those underpinning MOA or SAMOA, **Renoir** [12] is a high-performance dataflow framework written in Rust, offering zero-cost abstractions and cache-efficient memory management that make it particularly well-suited to the tight per-instance processing loop of ensemble SML. To address the limitations of existing centralized frameworks, this work introduces a horizontally parallelized architecture for Streaming Machine Learning (SML) built on the Renoir dataflow runtime. We successfully implement several ensemble methods based on decision trees [13], specifically Hoeffding-based ensembles like Adaptive Random Forest (ARF) [19] and Streaming Random Patches (SRP) [20], as well as Mondrian-based ensem-

bles like the Aggregated Mondrian Forest (AMF) [32]. In all these implemented models, ensemble diversity is injected and maintained by assigning different starting seeds to each base learner across the distributed workers. To our knowledge, this is the first system to expose horizontal parallelism for multiple ensemble SML algorithms within a unified, framework-level architecture.

6 | Conclusions

This thesis investigated the behavior of ensemble methods in Streaming Machine Learning (SML) under multithreaded environments, delivering both a high-performance engineering artifact and novel empirical insights into the computational boundaries of online learning. By analyzing established sequential frameworks such as Massive Online Analysis (MOA), River, and ONE-shot LEARNing (Onelearn), this work identified key architectural bottlenecks and addressed them by introducing a production-quality, fully validated, and generalized distributed streaming ensemble learning framework. Implemented in Rust and leveraging theRenoir dataflow runtime, the framework unifies three state-of-the-art ensemble paradigms—Adaptive Random Forest (ARF), Streaming Random Patches (SRP), and Aggregated Mondrian Forest (AMF)—into a single, highly optimized environment.

Initial accuracy evaluations successfully validated the fidelity of our implementations. The results confirmed that translating these complex algorithms from various languages into a Rust-based dataflow architecture preserves the predictive performance and theoretical guarantees of the original models.

In terms of performance, throughput benchmarking revealed that even when constrained to a single core, the Renoir-based implementation significantly outperforms existing SML frameworks, fully leveraging the raw execution speed and memory efficiency of the Rust language. Furthermore, the architecture successfully achieves horizontal scaling. While the speedup ratios eventually exhibit diminishing returns as the number of available cores increases, this behavior is largely dictated by underlying hardware limitations and inherent synchronization bottlenecks, rather than algorithmic flaws. Despite these architectural improvements, the evaluation highlighted that the curse of dimensionality remains a fundamental challenge, heavily penalizing traversal and structural performance as the number of dataset features grows.

The massive throughput gains unlocked by this distributed architecture enabled an extensive evaluation focused exclusively on scaling the number of trees within the ensembles. Processing such a high volume of estimators in a real-time stream was previously rendered intractable by the computational constraints of single-threaded execution.

However, our empirical findings demonstrate that scaling ensemble sizes by simply adding more trees yields a finite utility boundary. The statistical improvements in predictive accuracy rapidly reach a strict plateau, where adding more estimators provides negligible generalization gains. More importantly, because streaming environments demand sub-millisecond execution guarantees, the linear increase in memory footprint and processing latency introduced by an oversized forest quickly crosses a critical threshold. Beyond this point, the minor accuracy improvements fail to justify the massive throughput penalties, making fixed, tightly capped ensemble sizes an operational necessity for real-world deployments.

Crucially, this limitation does not diminish the value of horizontal parallelism; rather, it underscores the structural significance of the framework developed in this thesis. By decoupling the streaming pipeline and proving that horizontal scaling can comfortably maximize multi-core hardware, this work provides the definitive foundation for next-generation streaming architectures. Instead of wasting computational budgets on redundant trees that yield diminishing returns, the high-throughput pipeline allows systems to strategically reallocate exceeding, idle cores to high-value background tasks. As detailed in our future work section, these parallel resources can be immediately leveraged for multi-candidate drift tracking, real-time hyperparameter sweeps, and continuous model distillation. Consequently, this thesis shifts the paradigm of horizontal scaling in streaming machine learning from a simple quest for larger models to a sophisticated, multi-tenant orchestration of adaptive learning.

In conclusion, this thesis demonstrates that horizontal parallelism combined with rigid systems-level engineering is a highly effective paradigm for scaling ensemble methods in SML. The developed framework establishes a versatile, highly extensible, and production-ready baseline that bridges the gap between theoretical online learning and high-velocity distributed stream processing.

6.1. Future Works

The Renoir-based architecture developed in this thesis provides a highly extensible pipeline for streaming ensembles. Building upon this framework, we identify several primary directions for future research.

First, the Aggregated Mondrian Forest (AMF) algorithm requires optimization to address both structural and algorithmic limitations. Structurally, AMF demands memory compression or quantization techniques to reduce the heavy footprint of storing bounding box partitions. Algorithmically, it currently lacks an active forgetting mechanism, mean-

ing that integrating dynamic subtree pruning or explicit drift detection would allow it to adapt to severe concept drift without infinite memory growth.

Second, while this thesis proved the efficacy of horizontal scaling, real-world deployments often strictly cap ensemble sizes, such as limiting the forest to exactly ten trees, to balance speed and accuracy. To maximize multi-core utilization when horizontal scaling reaches these practical limits, future iterations should implement vertical parallelism by distributing the node-splitting or feature partitioning workload within the individual trees themselves.

Third, the current implementation of ARF and SRP maintains a single background learner on the same Renoir node. A natural extension is to offload background learner training to a dedicated distributed node, enabling true parallel drift recovery without competing for resources with the active ensemble. Going further, each tree could maintain multiple candidate background trees simultaneously, each trained with different hyperparameters, patch sizes, or feature subsets. Upon drift detection, the candidate with the best recent performance would be promoted, accelerating recovery and improving robustness. This multi-candidate strategy is specific to ARF and SRP, as AMF does not employ background learners.

Fourth, the framework is well suited for heterogeneous ensemble strategies, where multiple tree instances run in parallel with deliberately varied configurations, both for base learner types and hyperparameters, and their predictions are aggregated. This approach diversifies the ensemble’s inductive bias and can yield meaningful accuracy gains at constant ensemble size. The same infrastructure also enables online hyperparameter tuning: by evaluating configurations against a fading holdout window, the system can automatically promote the best-performing variant and adapt parameters such as patch size or ensemble size over time without manual intervention.

Finally, the framework must be extended to provide native support for categorical and boolean data. Currently, these data types must be transformed via one-hot encoding, which artificially expands the feature space. This expansion actively worsens the curse of dimensionality and degrades tree traversal performance. Implementing native categorical splitting rules directly inside the decision nodes would keep the feature space compact, thereby significantly boosting both processing throughput and overall memory efficiency. A complementary long-term direction is ensemble distillation: running a lightweight student model in the background that continuously learns to approximate the ensemble’s predictions, yielding a compact, deployment-friendly model without sacrificing the accuracy accumulated during online training.

Bibliography

- [1] Comparing models. In *Applied Machine Learning for Tobacco Control (AML4TD)*. URL <https://aml4td.org/chapters/comparing-models.html>. Accessed: June 2026.
- [2] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, spring joint computer conference*, pages 483–485. AFIPS, 1967.
- [3] Y. Ben-Haim and E. Tom-Tov. A streaming parallel decision tree algorithm. *J. Mach. Learn. Res.*, 11:849–872, Mar. 2010. ISSN 1532-4435.
- [4] A. Bifet and R. Gavaldà. Learning from time-changing data with adaptive windowing. volume 7, 04 2007. doi: 10.1137/1.9781611972771.42.
- [5] A. Bifet and R. Gavaldà. Adaptive learning from evolving data streams. pages 249–260, 08 2009. ISBN 978-3-642-03914-0. doi: 10.1007/978-3-642-03915-7_22.
- [6] A. Bifet, G. Holmes, B. Pfahringer, P. Kranen, H. Kremer, T. Jansen, and T. Seidl. Moa: Massive online analysis, a framework for stream classification and clustering. In T. Diethe, N. Cristianini, and J. Shawe-Taylor, editors, *Proceedings of the First Workshop on Applications of Pattern Analysis*, volume 11 of *Proceedings of Machine Learning Research*, pages 44–50, Cumberland Lodge, Windsor, UK, 01–03 Sep 2010. PMLR. URL <https://proceedings.mlr.press/v11/bifet10a.html>.
- [7] A. Bifet, R. Gavaldà, G. Holmes, and B. Pfahringer. *Machine Learning for Data Streams: with Practical Examples in MOA*. The MIT Press, 03 2018. ISBN 9780262346047. doi: 10.7551/mitpress/10654.001.0001.
- [8] L. Breiman, J. Friedman, and C. J. Stone. *Classification and regression trees*. Chapman and Hall/CRC, 2017.
- [9] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas. Apache flink: Stream and batch processing in a single engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 38(4):28–38, 2015.

- [10] N. Cesa-Bianchi and G. Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, New York, NY, USA, 2006. ISBN 978-0-521-84146-7. doi: 10.1017/CBO9780511546921.
- [11] D. Christian. Scaling stream-based decision trees on a distributed dataflow engine. Master’s thesis, Politecnico di Milano, 2 2026.
- [12] L. De Martini, A. Margara, G. Cugola, M. Donadoni, and E. Morassutto. The renoir dataflow platform: Efficient data processing without complexity. *Future generation computer systems*, 160:472–488, 2024.
- [13] M. Denil, D. Matheson, and N. de Freitas. Consistency of online random forests. In *International Conference on Machine Learning (ICML)*, pages 1256–1264. PMLR, 2013.
- [14] T. G. Dietterich. Ensemble methods in machine learning. In *International Workshop on Multiple Classifier Systems*, pages 1–15. Springer, 2000.
- [15] P. Domingos and G. Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’00, page 71–80, New York, NY, USA, 2000. Association for Computing Machinery. ISBN 1581132336. doi: 10.1145/347090.347107.
- [16] J. Gama. *Knowledge Discovery from Data Streams*. Chapman & Hall/CRC, 1st edition, 2010. ISBN 1439826110.
- [17] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia. A survey on concept drift adaptation. *ACM Comput. Surv.*, 46(4):44:1–44:37, 2014.
- [18] J. a. Gama, R. Sebastião, and P. P. Rodrigues. On evaluating stream learning algorithms. *Mach. Learn.*, 90(3):317–346, Mar. 2013. ISSN 0885-6125. doi: 10.1007/s10994-012-5320-9. URL <https://doi.org/10.1007/s10994-012-5320-9>.
- [19] H. M. Gomes, J. Read, and A. Bifet. Adaptive random forests for evolving data stream classification. pages 240–249, 10 2017. doi: 10.1007/s10994-017-5642-8.
- [20] H. M. Gomes, J. Read, and A. Bifet. Streaming random patches for evolving data stream classification. pages 240–249, 11 2019. doi: 10.1109/ICDM.2019.00034.
- [21] H. M. Gomes, J. Read, A. Bifet, J. P. Barddal, and J. a. Gama. Machine learning for streaming data: state of the art, challenges, and opportunities. *SIGKDD Explor. Newsl.*, 21(2):6–22, Nov. 2019. ISSN 1931-0145. doi: 10.1145/3373464.3373470. URL <https://doi.org/10.1145/3373464.3373470>.

- [22] P. F. Harrison and N. M. Cook. Biased bootstrap sampling for efficient two-sample testing. *JINST*, 13(12):P12014, 2018. doi: 10.1088/1748-0221/13/12/P12014.
- [23] T. Hastie, R. Tibshirani, and J. Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2 edition, 2009.
- [24] N. Kourtellis, G. D. F. Morales, A. Bifet, and A. Murdopo. VHT: Vertical Hoeffding Tree, 2016.
- [25] M. Kuhn and K. Johnson. Applied Machine Learning for Tabular Data, 2023. URL <https://aml4td.org>.
- [26] D. Lakens. *Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses*. Soc Psychol Personal Sci, 2017. doi: 10.1177/1948550617697177.
- [27] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar. Mllib: Machine learning in apache spark. *Journal of Machine Learning Research*, 17(34):1–7, 2016.
- [28] J. Montiel, J. Read, A. Bifet, and T. Abdessalem. Scikit-multiflow: A multi-output streaming framework. *Journal of Machine Learning Research*, 19(72):1–5, 2018.
- [29] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, et al. River: machine learning for streaming data in python. 2021.
- [30] G. D. F. Morales and A. Bifet. SAMOA: Scalable advanced massive online analysis. *Journal of Machine Learning Research*, 16:149–153, 2015.
- [31] J. Mourtada, S. Gaïffas, and E. Scornet. Universal consistency and minimax rates for online mondrian forests. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [32] J. Mourtada, S. Gaïffas, and E. Scornet. Amf: Aggregated mondrian forests for online learning. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 83(3):505–533, 2021.
- [33] J. Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann series in machine learning. Elsevier Science, 1993. ISBN 9781558602380.
- [34] Y. W. Teh. *Dirichlet Process*. Springer US, Boston, MA, 2017. ISBN 978-1-4899-

- 7687-1. doi: 10.1007/978-1-4899-7687-1_219. URL https://doi.org/10.1007/978-1-4899-7687-1_219.
- [35] A. Toshniwal, S. Taneja, A. Shukla, K. Ramasamy, J. M. Patel, S. Kulkarni, J. Jackson, K. Gade, M. Fu, J. Donham, N. Bhagat, S. Mittal, and D. Ryaboy. Storm@twitter. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, pages 147–156. ACM, 2014.
- [36] A. Tsymbal. The problem of concept drift: Definitions and related work. 05 2004.
- [37] F. M. Willems, Y. M. Shtarkov, and T. J. Tjalkens. Reflections on "the context-tree weighting method: Basic properties". *IEEE Information Theory Society Newsletter*, 47(1), March 1997.

List of Figures

2.1	Local and global feature subset randomization	16
2.2	Left: A single guillotine cut in the 2D feature space. Right: The corresponding tree structure.	18
3.1	High-level class diagram of the learner abstraction.	30
3.2	Logical stages of the ensemble pipeline.	36
3.3	Renoir pipeline: Input and Initialization	36
3.4	Renoir pipeline: Per-Tree Processing	37
3.5	Renoir pipeline: Per-Instance Aggregation	37
3.6	Renoir pipeline: Output	38
4.1	Accuracy over time: notable results.	53
4.2	Scaling behaviors: notable results.	63
4.3	Speed up behaviors: notable results.	65
4.4	Speedup comparison between MOA and Renoir: notable results.	66

List of Tables

2.1	Comparison of ARF and SRP characteristics	17
3.1	Global configuration parameters.	31
3.2	Algorithm-specific configuration hyperparameters.	32
4.1	Summary of dataset characteristics	43
4.2	ARF and SRP parameters	50
4.3	Hoeffding Tree-Based Ensembles accuracy (%) using 10 base models	50
4.4	Hoeffding Tree-Based Ensembles accuracy (%) using 100 base models . . .	51
4.5	Hoeffding Tree-Based Ensembles throughput (instances/second) using 10 base models and 1 core	52
4.6	Hoeffding Tree-Based Ensembles throughput (instances/second) using 100 base models and 32 cores	52
4.7	River vs Onelearn throughput with 10 trees	54
4.8	AMF parameters	55
4.9	AMF accuracy (%) using 10 base models	56
4.10	AMF throughput (instances/second): 10 base models and 1 core	57
4.11	ARF throughput efficiency in remote execution using 100 base models and 32 cores	59
4.12	AMF throughput efficiency in remote execution using 20 base models and 32 cores	60
4.13	Efficiency (%) for ARF and SRP, 100-tree ensemble.	63
4.14	Speedup Ratios for AMF, 20-tree ensemble	65
4.15	Scalability efficiency of ARF (%)	66
4.16	VHT comparison with ARF using 100 base models and 32 cores	70
4.17	Comparison of throughput speedup factors when transitioning from MOA to Renoir for VHT and ARF.	71
4.18	Performance metrics (Accuracy and Throughput) across different hyperpa- rameter configurations for Internet ads and LED(g) - ARF	75

4.19 Trade-off Evaluation: Performance vs. Relative Gains Across Ensemble
 Sizes - AMF 77

Acknowledgments

Oggi, 22 luglio 2026, scalo l'ultimo gradino di una lunghissima e tortuosa scalata, ricca di alti e bassi, momenti significativi e periodi di sconforto, in cui ho avuto però il privilegio di non essere mai solo. Sono profondamente grato a tutte le persone che mi hanno sostenuto, in svariati modi, nel raggiungere questa importante pietra miliare della mia vita personale e professionale.

La mia famiglia occupa sicuramente il primo posto: sempre presente, di supporto e con un'enorme capacità di credere in me in ogni circostanza e di fronte a qualsiasi difficoltà, senza farmi mai mancare anche il fondamentale sostegno economico. Mio padre Massimo, sempre fiero di me, che deve far sapere al mondo intero di avere un figlio ingegnere, e capace di dare il giusto peso alle avversità; mia madre Giusi, che non ha mai dubitato delle mie capacità, anche quando avrebbe invece potuto; e i miei fratelli Gabriele e Francesco, per i quali so di essere stato un esempio da seguire e che ho sempre sentito sinceramente felici dei miei traguardi. Un pensiero inoltre va a zia Tania, persona splendida che, anche se a distanza, mi ha sempre sostenuto facendomi sentire compreso; a zio Francesco, a nonna Rosa, a zia Valentina e al neo-arrivato Edoardo, a cui spero di poter far leggere questa tesi un giorno.

Colonna portante che mi ha supportato dal primo all'ultimo giorno è la bellissima Emily, pronta a offrirmi un sostegno emotivo incrollabile anche di fronte alle mie più piccole debolezze. Con te ho potuto bilanciare i ritmi a tratti assurdi del Politecnico con la bellezza della vita quotidiana: le uscite, i pisolini, la capacità di staccare la spina e vivere il mondo. Con te ho vissuto tre anni splendidi, pieni di emozioni e caratterizzati da una sicurezza e una stabilità che mi hanno permesso di non crollare mai. Con te ho trovato un vero tesoro. Grazie con tutto il cuore. Un pensiero va anche alla Lily piccolina, arrivata di recente: minuscola, monellina ed estremamente graziosa, capace di regalare tanto affetto e di farmi dimenticare ogni preoccupazione con un solo sguardo.

Vorrei poi ringraziare il Prof. Margara, il Prof. Cugola, Luca, Dario, Federico e l'intero gruppo di ricerca per il loro fantastico e costante supporto durante tutto il periodo di tesi. Ho apprezzato infinitamente la cura e l'interesse dimostrati non solo verso il progetto, ma

verso gli studenti stessi; questo approccio mi ha dato la possibilità di imparare moltissimo e, contemporaneamente, di conoscere persone splendide, scambiare idee e vivere i miei ultimi mesi universitari con il giusto equilibrio.

Tutto questo, però, non sarebbe stato lo stesso senza i miei colleghi universitari, ormai diventati grandi amici, insieme ai compagni di una vita. Con loro ho condiviso ore di lezioni incomprensibili, appunti, centinaia di esercizi e intere giornate (e nottate) in compagnia.

Christian, con i suoi impeccabili appunti in L^AT_EX, la disponibilità h24, la gentilezza nel rispiegarmi un intero corso da zero a due settimane dall'esame, il costante supporto nella ricerca del lavoro ideale, e le immancabili serate di gioco a casa sua... grazie, sei stato senza dubbio la figura più rilevante di questi ultimi anni. E, ovviamente, non posso dimenticare Lucky e Simona. Filippo e Anna, con cui ho costruito un legame che va ben oltre l'ambito accademico, condividendo arancini, granite, poker, e la certezza che non ci perderemo mai di vista. Carlo e il mitico, mistico Balvinder: menti geniali e persone dal cuore d'oro, sempre pronti a rispiegare i concetti più complessi nel modo più intuitivo e accessibile possibile. Enrico, Rebecca, Giulio, Francesco, Amanda, Cassy, Greta, Alessandro, Stefano... siete tantissimi, e per ognuno di voi avrei aneddoti, storie e ringraziamenti speciali da fare.

Ci tengo a fissare queste brevi parole in un lavoro che rappresenta il culmine di una fase della mia vita e l'inizio di un'altra, per mantenere vivo il ricordo di tutte le persone fantastiche che con me hanno pianto, riso e gioito, e a cui sarò per sempre grato.

Grazie di cuore, a voi e a tutti coloro che non ho citato esplicitamente: è anche merito vostro se termino questo difficile percorso con il sorriso, guardando con serenità non solo al futuro, ma anche al passato.